

$$Cr = 128 + 0.44R - 0.37G + 0.07B$$

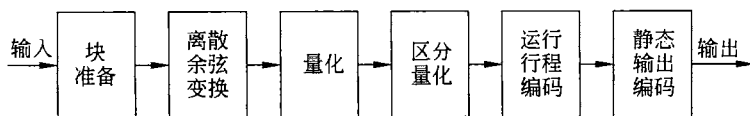


图 7-44 JPEG 有损顺序编码的步骤

为 Y、Cb 和 Cr 分别构造一个独立的矩阵。接下来，在 Cb 和 Cr 矩阵中，针对每四个像素组成的方块计算其平均值，以便将矩阵减小至 320×240 。这种缩减是有损的，但是眼睛几乎不会注意到，因为眼睛对亮度比对色度更加敏感。然而，这样做却能够把整个数据量压缩一半。现在从这三个矩阵的每个元素中减去 128，以便使元素范围的中间值变成 0。最后，每个矩阵被分割成 8×8 的块。Y 矩阵有 4800 个块；其他两个矩阵有 1200 个块，如图 7-45(b)所示。

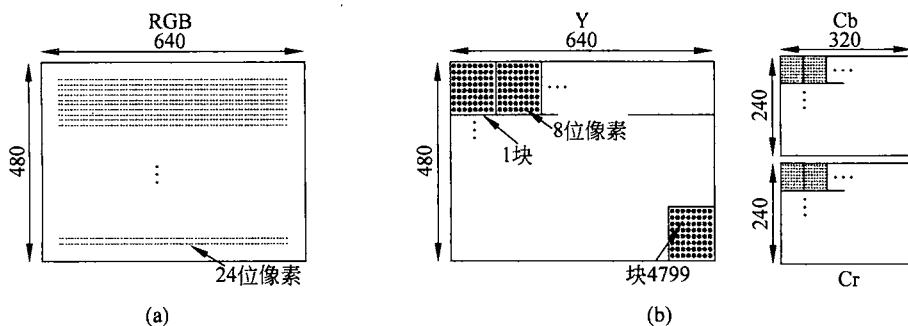


图 7-45

(a) RGB 输入数据; (b) 块准备之后

JPEG 的第 2 步是对 7200 个块中的每一个块单独应用离散余弦变换 (DCT, Discrete Cosine Transformation)。每个 DCT 的输出是一个 8×8 的 DCT 系数矩阵。DCT 元素(0, 0)是每个块的平均值。其他的元素表明了在每个空间频率处的频谱功率。通常，随着元素离原点(0, 0)的距离越来越远，它们的值也迅速地减小，如图 7-46 所示。

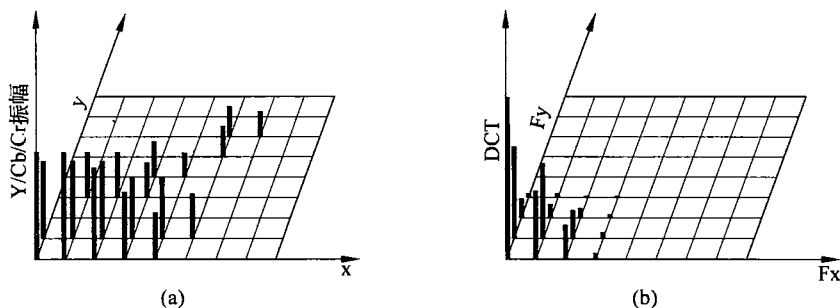


图 7-46

(a) Y 矩阵的一块; (b) DCT 系数

一旦完成 DCT, JPEG 编码进入第 3 步, 这一步称为量化 (quantization)。在这一步中, 一些不那么重要的 DCT 系数被除掉。这种 (有损) 变换是这样来完成的: 将 8×8 DCT 矩阵中的每个系数除以一个权值, 而该权值是通过查询一张表获得的。如果所有的权值都是

1，则变换实际上没有做任何事情。然而，如果权值表从原点开始迅速递增，则较高的空间频率将被快速丢弃。

图 7-47 给出了这一步的一个例子。在这里，我们可以看到初始的 DCT 矩阵、量化表，以及每个 DCT 元素除以量化表中对应元素之后得到的结果。量化表中的值不是 JPEG 标准的一部分。每个应用必须提供它自己的量化表，从而允许它自己来控制损失-压缩之间的权衡。

DCT系数								量化表								C量化后的系数							
150	80	40	14	4	2	1	0	1	1	2	4	8	16	32	64	150	80	20	4	1	0	0	0
92	75	36	10	6	1	0	0	1	1	2	4	8	16	32	64	92	75	18	3	1	0	0	0
52	28	26	8	7	4	0	0	2	2	2	4	8	16	32	64	26	19	13	2	1	0	0	0
12	8	6	4	2	1	0	0	4	4	4	4	8	16	32	64	3	2	2	1	0	0	0	0
4	3	2	0	0	0	0	0	8	8	8	8	8	16	32	64	1	0	0	0	0	0	0	0
2	2	1	1	0	0	0	0	16	16	16	16	16	16	32	64	0	0	0	0	0	0	0	0
1	1	0	0	0	0	0	0	32	32	32	32	32	32	32	64	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	64	64	64	64	64	64	64	64	0	0	0	0	0	0	0	0

图 7-47 量化 DCT 系数的计算

第 4 步将每一块的(0, 0)元素的值用它与前一块中对应元素的差值来替代。因为这些元素是它们各自块的平均值，它们应该变化得很缓慢，因此，采用差值之后可以把它们中的大部分减成很小的数值。其他的元素不计算差值。

第 5 步将 64 个元素排列起来，并且对此列表使用行程编码方法。按照从左到右再从上往下的方法对块进行扫描，这种方式不会将 0 集中到一起，所以 JPEG 采用了“Z”字形的扫描模式，如图 7-48 所示。在这个例子中，“Z”字形扫描模式在矩阵末端产生 38 个连续的 0。这个 0 串能够被减少为一个计数值，即由计数值来表明这 38 个 0，这项技术即为行程编码（run-length encoding）。

150	80	20	4	1	0	0	0
92	75	18	3	1	0	0	0
26	19	13	2	1	0	0	0
3	2	2	1	0	0	0	0
1	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

图 7-48 量化之后值的发送次序

现在我们有了一系列数值来代表图像（在变换空间中）。第 6 步对这些数值进行霍夫曼编码，以便于存储或传输；给常见的数值分配较短的编码，即不常见的数值被分配较长的编码。

JPEG 或许看起来很复杂，它的确如此，因为它确实复杂。然而，能够达到 20：1 的压缩效率也是物有所值。解码 JPEG 图像需要运行后向算法。JPEG 算法基本上是对称的：解码需要和编码花一样长的时间。正如我们将看到的那样，并不是所有的压缩算法都具有这种特性。

MPEG 标准

最后，我们来到问题的核心：运动图像专家组（MPEG，Motion Picture Experts Group）标准。虽然研究者开发出了许多专用的算法，但这些标准定义了压缩视频使用的主要算法。它们从 1993 年开始已经成为国际标准。因为电影同时包含了图像和声音，所以，MPEG 既可以压缩音频同时也可以压缩视频。我们已经讨论过音频压缩和静止图像的压缩，现在来讨论视频压缩。

标准 MPEG-1 (它包括了 MP3 音频) 首次发布于 1993 年, 从此获得了广泛的应用, 直到今天依然如此。它的目标是产生录像机品质的输出, 而这个输出必须被压缩 40:1 才能达到 1 Mbps 左右的速率。这样的视频适应用户通过 Internet 广泛使用 Web 站点。如果你不记得视频录像机也不必焦虑——如果有的话, MPEG-1 可用来在 CD 上存储电影。如果你连 CD 是什么也不知道, 我们将很快转移到 MPEG-2。

标准 MPEG-2 发布于 1996 年, 它的设计目标是压缩广播品质的视频。现在它已非常流行, 因为它是 DVD 视频编码 (这是通向 Internet 无法回避的方式) 和数字广播电视 (作为 DVB) 的基础。DVD 品质的视频一般的编码速率是 4~8 Mbps。

MPEG-4 标准有两种视频格式。第一种格式发布于 1999 年, 以基于对象的表示来编码视频。它允许自然图像和合成图像, 以及其他各类媒体的混合, 例如, 一个气象预报员站在气候地图之前。有了这种结构, 很容易地让程序与电影数据交互。第二种格式于 2003 年发布, 称为 H.264 或高级视频编码 (AVC, Advanced Video Coding)。它的目标是编码率为早期相同品质水平的视频编码率的一半, 以便更好地支持通过网络进行视频传输。这个编码器用于大多数蓝光光盘的 HDTV。

所有这些标准的细节非常多而且种类繁多。后来的标准相比以前的标准有许多更多的功能和编码选项。不过, 我们不会详谈这些细节。在大多数情况下, 随着时间的推移, 视频压缩的收益都来自许多小的改进, 而在如何压缩视频的方法上没有根本性的转变。因此, 我们将描绘整体概念。

MPEG 能同时压缩音频和视频。由于音频和视频编码器独立工作, 这就存在一个如何在接收端同步两个流的问题。解决的办法是用一个时钟, 将当前时间的时间戳输出到两个编码器。这些时间戳被包含在编码后的输出流中, 并且一路传播到接收端, 接收端的播放器可以用这些时间戳来同步音频流和视频流。

MPEG 视频压缩利用了电影中存在的两种冗余优势: 空间和时间。通过用 JPEG 对每一帧单独进行简单编码可以获得空间冗余。这种方法偶尔会被使用, 就像编辑视频制作那样, 特别是当需要随机访问每一帧时。在这种模式下, 可以达到 JPEG 的压缩水平。

还有一种现象也可以考虑用来压缩图像。连续的帧往往几乎是相同的, 充分利用这样的事实还可以获得更大的压缩率。其效果比第一次出现的要差一些, 因为许多电影导演在剪辑电影时差不多每隔 3 秒或 4 秒就切换场景 (试试计时一个电影片段, 并且对场景计数)。然而, 运行 75 个甚至更多个高度相似的帧比简单地利用 JPEG 对每一帧进行独立编码提供了更多的冗余, 因而具有更大的压缩潜力。

考虑这样的场景: 摄像机和背景都固定不变并且只有一两个演员慢慢地走来走去, 帧和帧之间几乎所有的像素都是相同的。在这里, 只要从前一帧中减去当前帧, 并且对两帧之差运行 JPEG 算法, 效果就会非常好。然而, 对于那些场景: 摄像机在移动或者不断地推拉, 这项技术的效果会很差。这就需要某种办法来补偿这种运动。这恰好是 MPEG 擅长的的工作, 也是 MPEG 和 JPEG 的主要区别。

MPEG 的输出包括 3 类帧:

- (1) I-帧 (Intracoded frames): 帧内编码帧包含了压缩的静止图片。
- (2) P-帧 (Predictive frames): 预测帧是与前一帧的逐块差值。
- (3) B-帧 (Bidirectional frames): 双向帧是与前一帧和后一帧的逐块差值。

I-帧只是一些静止图片。它们采用 JPEG 或者其他类似的算法进行编码。在输出流中周期性地出现 I-帧(比如,每秒 1 或者 2 次)是有价值的,其中的原因有三。第一, MPEG 可能被用于组播传输,观众可以随意收看视频节目。如果所有的帧都依赖于前面的帧,那么一直要回溯到第一帧,否则错过第一帧的人就永远也解不出任何一个后续帧。第二,如果任意一帧发生了接收错误,则后续帧的解码将再也不可能进行:从那时开始的一切都将不知所云。第三,若没有 I 帧,那么,在快进或者回退时,解码器将不得不计算经过的每一帧,以便知道暂停在某个帧的完整值。因此 I-帧必须周期性地出现在媒体流中。

相反, P-帧是对帧间的差值进行编码。P 帧建立在宏块(macroblock)想法的基础上,宏块覆盖了亮度空间中的 16×16 像素,以及色度空间中的 8×8 像素。通过搜索前一帧中与自己相同的部分或稍有不同的部分来进行宏块的编码。

图 7-49 显示了 P 帧用法的一个例子。在这里,我们看到 3 个连续的帧,它们具有相同的背景,但是人的位置有所不同。包含这 3 个帧背景场景的宏块能够精确地匹配,但是包含人的宏块在位置上有一定的未知量偏移,而且必须被跟踪。

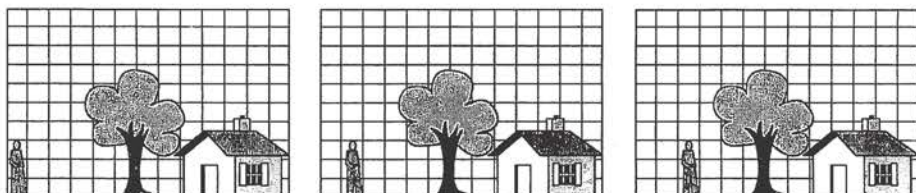


图 7-49 3 个连续帧

MPEG 标准并没有规定如何进行搜索、搜索的范围多大,也没有规定如何评价一个匹配的好坏。这些都由算法的每个实现自行决定。例如,一个实现可能是这样的:在前一帧的当前位置上搜索一个宏块,并且也在 x 方向上偏移 $\pm \Delta x$ 范围内和 y 方向偏移 $\pm \Delta y$ 范围内的所有位置上搜索该宏块。对于每个位置,计算亮度矩阵的匹配数量。得分值最高的那个位置只要超过某个预定的阈值,则它被认为胜出者。否则,该宏块称为已丢失。当然,还有可能存在其他更加复杂的算法。

如果找到了一个宏块,则利用它与前一帧中对应宏块之间的差值进行编码(包括亮度和两个色度)。然后,对这些差值矩阵进行离散余弦变换、量化、行程编码和霍夫曼编码,跟通常的做法一样。再者,在输出流中该宏块的值就是运动向量(相对于前一个位置在每个方向上运动了多远),紧随其后的是它差值的编码。如果在前一帧中没有定位到该宏块,则对当前值进行编码,就如同 I-帧的做法一样。

很明显,这个算法是高度非对称的。只要自己愿意,任何一个实现都可以任意试探前一帧中每一个合理的位置,不顾一切地找出每一个刚过去的宏块,无论该宏块移动到哪里。这种做法以极慢的编码速度为代价,换来最小化 MPEG 输出流。对于电影资料库的一次性编码应用来说,这种方法非常好,但是对于实时视频会议来说则非常可怕。

类似地,每个实现可以自由决定什么组成了“找到的”宏块。这种自由度允许各个算法在质量和速度上进行竞争,但总是会产生兼容的 MPEG 输出。

到目前为止,对 MPEG 的解码非常直截了当。解码 I-帧类似于解码 JPEG 图像。解码 P-帧则要求解码器缓冲前面的帧,然后利用完全编码的宏块以及包含与前一帧差值的宏块,

在另一个缓冲区中构造出新的一帧。这个新帧由一个宏块接着一个宏块组合起来。

B-帧类似于 P-帧，不同之处在于，它允许参照的宏块可以是前一帧，也可以是后一帧。这种更大的自由度带来了更强的运动补偿，例如，当有些物体在其他物体的前面、后面或者左右通过时，这种方法非常有用。为了进行 B-帧解码，解码器需要同时在内存中保留一系列帧：过去的帧、当前正在被解码的帧以及未来的帧。同样，尽管 B-帧的解码更复杂，而且增加了某些额外的延迟。这是因为给定一个 B-帧不能马上被解码出来，必须要等到它依赖的后续帧到达。因此，尽管 B-帧提供了最好的压缩算法，由于其更强的复杂性以及对缓冲区的需求，并不是总能适用于一切场合。

MPEG 标准包含许多增强型功能，这些技术实现了优良的压缩级。AVC 可以超过 50:1 的压缩率来压缩视频，从而把网络带宽的需求减少到相当的程度。有关 AVC 的更多信息，请参阅 (Sullivan 和 Wiegand, 2005)。

7.4.3 流式存储媒体

现在，让我们把注意力转移到网络应用。我们的第一种情况针对早已存储在文件中的流媒体。最常见的例子是在 Internet 上观看视频。这是视频点播 (VoD, Video on Demand) 的一种形式。其他形式的视频点播使用了服务提供商网络来传送影片，这种网络通常独立于 Internet (例如，有线电视网络)。

在下一节，我们将考查如何现场直播流媒体，例如，广播 IPTV 和 Internet 广播。然后我们将考查第三种情况的实时会议，比如语音 IP 电话或 Skype 视频会议。这三种情况都通过网络提供音频和视频，但对网络的需求越来越严格，我们必须越来越多地关注延迟和抖动。

Internet 上充满了音乐和视频站点，这些站点流化存储在其上的媒体文件。实际上，处理存储媒体的最简单方式不是流化它。想象一下，你想要创建一个在线电影租赁站点，与苹果的 iTunes 竞争。常规的站点允许用户下载视频，然后再观看该视频 (当然，是在付费之后)。这个步骤序列如图 7-50 所示，我们把它们拼凑出来，以便与下一个例子作对比。

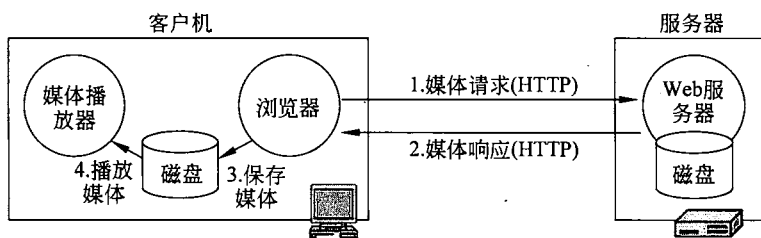


图 7-50 通过简单的下载在 Web 上播放媒体

当用户单击一个电影时浏览器开始行动了。在第 1 步中，浏览器给该电影链接的 Web 服务器发送一个 HTTP 请求，请求该电影。接着第 2 步，服务器从磁盘上获取该电影 (只是一个 MP4 或其他某种格式的文件) 并将它发回浏览器。使用 MIME 类型，例如 video/mp4，浏览器查找应该如何显示此文件。在这种情况下，它是一个媒体播放器，显示为辅助应用程序，尽管它可能也是一个插件。浏览器将整个电影保存到磁盘上的一个临时文件中。然

后，浏览器启动媒体播放器，并将临时文件的名字传递给它。最后，在第4步中，媒体播放器开始读取文件并播放该电影。

原则上，这种方法完全正确，它将播放出电影。这里没有任何实时网络问题需要处理，因为下载电影只是一次简单的文件下载。唯一的麻烦是在播放电影之前，必须通过网络传输整个电影。大多数客户恐怕都不愿意为他们的“视频点播”等上一个小时。因此这种模式可能会产生问题，甚至对音频也一样。想象一下，你在购买一张唱片之前想试听一首歌曲。如果歌曲是4 MB，这是一首典型的MP3歌曲大小，而且宽带连接为1 Mbps，那么用户将遭遇半分钟的静谧，然后才能开始试听。这样的模式不太可能卖出许多唱片。

为了避免这个问题而且不改变浏览器的工作方式，站点可以使用如图7-51的设计。链接到电影的页面不是实际的电影文件。相反，它是一个称为元文件（metafile）的文件。元文件是一种非常简短的文件，仅仅给出了电影的名字（可能还有其他关键描述符）。一个简单的元文件可能只有一行ASCII文本，看上去像这样：

rtsp://joes-movie-server/movie-0025.mp4

浏览器像往常一样得到这个页面，现在是只有一行长的文件，如图中第1步和第2步所示。然后，它启动媒体播放器，并将这一行长的文件传递给播放器，如第3步所示，一切照常进行。媒体播放器读取元文件，发现可以获得电影的一个URL。接着，播放器与joes-movie-server联系，并向服务器请求该电影，见第4步。然后，该电影被流式发回给媒体播放器，见第5步。这种安排的优点在于媒体播放器很快就能启动，仅仅在那个非常短的元文件下载后即可启动。一旦启动后，浏览器就再也不出现在循环中。媒体被直接发给播放器，在整个文件被下载之前就可以播放电影了。

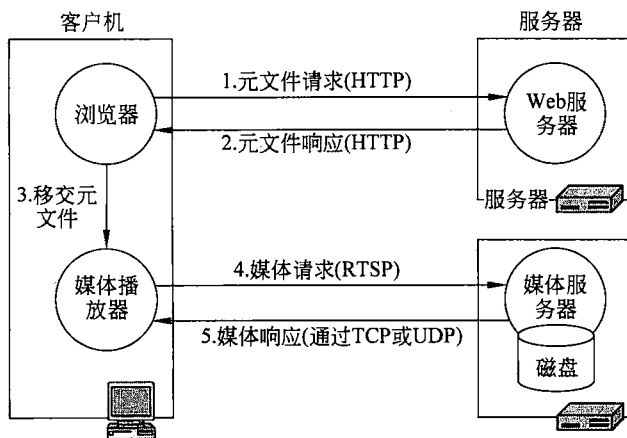


图 7-51 使用了 Web 和媒体服务器的流式媒体

在图 7-51 中，我们展示了两个服务器，因为元文件中给出的服务器通常与 Web 服务器不是同一个。事实上，它一般就不是 HTTP 服务器，而是一个专门的媒体服务器。在这个例子中，媒体服务器使用了实时流协议（RTSP，Real Time Streaming Protocol），正如方案名 RTSP 所指示的那样。

媒体播放器主要完成以下 4 项工作：

(1) 管理用户界面。

- (2) 处理传输错误。
- (3) 解压缩内容。
- (4) 消除抖动。

现在大多数媒体播放器都有华丽的用户界面，有的甚至模拟成一个立体声音响，面板上带有各种按钮、旋钮、调节滑块和可视显示窗等。通常播放器的面板膜样可以自由被替换成其他膜样，这样的面板称为皮肤 (skin)，用户可以将它拖放到播放器上。媒体播放器必须管理所有这些事项，并且与用户实行交互。

其他的几项工作与网络协议相关，并且取决于网络协议。我们将依次讨论每一个，首先从处理传输错误开始。究竟怎么处理传输错误依赖于用什么协议来传输媒体，是像 HTTP 那样基于 TCP 的传输协议，还是像 RTP 那样基于 UDP 的传输协议。这两种传输协议都可以实际使用。如果使用的是一个基于 TCP 的传输协议，那么媒体播放器不需要纠正错误，因为 TCP 通过使用重传机制已经提供了可靠性。这是一种简单的处理错误方法，至少对媒体播放器是这样，但它在后面的消除抖动步骤中很复杂。

另外，像 RTP 这种基于 UDP 的传输协议，可以用来移动数据。我们在第 6 章讨论过这点。使用这些协议，没有重传机制。因此，由于网络拥塞或传输错误，造成数据包的丢失将意味着一些媒体数据传不到播放器。这个问题要由媒体播放器来处理。

让我们来了解将要面对的困难。因为客户不喜欢他们的歌曲或电影出现较大的缺口，因此丢包是个问题。然而，丢包的损失又不像传输一个普通文件出现丢包时那么严重，因为少量媒体的丢失不需要降低用户的播放。对于视频，用户不太可能注意到偶尔每秒只出现 24 个新帧，而不是 25 个新帧。对于音频，播放期间的短暂间隙可以用时间接近的声音来掩盖。用户不可能发现这种替换，除非他们非常非常关注。

然而，上述推理的关键在于音频或者视频流中的缺口很小。网络拥塞或传输错误通常会导致整个数据包的丢失，并且丢包往往呈现少量的突发。面对这种问题，有两种策略可以用来降低数据包丢失对媒体丢失的影响：FEC 编码和交错编码。我们将依次描述每种策略。

前向纠错 (FEC, Forward Error Correction) 适用于应用程序级的简单纠错编码，我们已经在第 3 章学习过。整个数据包的奇偶校验提供了一个范例 (Shacham 和 McKenny, 1990)。每发送 4 个数据包，就据此构造第 5 个奇偶包，组成一组一起发送。如图 7-52 所示中的数据包 A、B、C 和 D。奇偶包 P 包含了冗余位，这些冗余位取自 4 个数据包中每个数据位的奇偶或“异或”和。我们希望对于大多数 5 个包组成的组中所有数据包都能到达接收端。如果果真如此，那么接收器只需简单地丢弃奇偶包。否则，如果只是丢失了校验包，也没有什么损害。

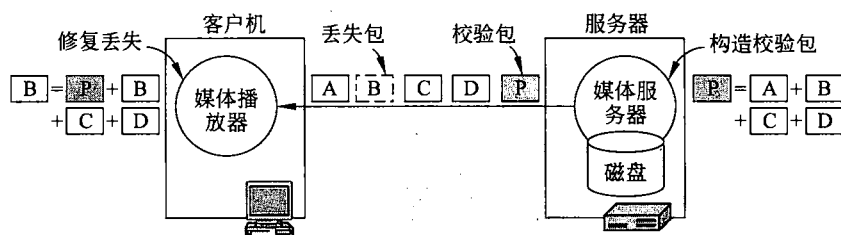


图 7-52 使用奇偶校验包来修复丢失包

然而，有时在传输过程中可能会丢失一个数据包，如图 7-52 中的 B 数据包。媒体播放器只接收到了三个数据包 A、C 和 D，加上奇偶包 P。通过设计，丢失那个数据包中的数据位可以从校验位重构出来。具体而言，假设“+”代表“异或”或模 2 加，那么利用异或特征（即 $X + Y + Y = X$ ），可重构出 B， $B = P + A + C + D$ 。

FEC 可以通过修复一些丢失的数据包，来减缓媒体播放器的损失程度，但它也只能起到一定程度的作用。如果在 5 个一组的包中丢失了两个包，那我们无论做什么都无法恢复出丢失的数据。需要注意的 FEC 另一个特征是为获得这种保护措施，我们必须支付的成本。每四个包必须生成第五个包，所以带宽的需求量比媒体所需要的多 25%。并且，还增加了接收端解码的延迟，因为我们可能需要等到奇偶包的到达，才可以重新建造出这个奇偶包之前丢失的数据包。

在上面所述的技术中还有一个聪明的技巧。第 3 章中我们描述了奇偶校验提供的是错误检测。而在这里，我们用奇偶校验进行纠错。怎样才能做到这点呢？答案是在这种情况下已经知道丢失的数据包。丢失的数据称为擦除（erasure）。在第 3 章中，当我们认为接收到的一个数据帧中某些位出错时，我们不知道究竟是哪些位出现了差错。这种情况比擦除情况更难处理。因此，有了擦除信息，奇偶校验就具备了纠错功能，而如果没有擦除信息，奇偶校验就只能提供错误检测功能。当我们讨论组播情景时，很快就能看到奇偶校验带来的另一个意想不到的好处。

第二种策略是所谓的交错编码（interleaving）。这种方法的基本原理是这样的：在传输之前把媒体的顺序混合或交错编码起来，然后在接收端拆混或解开交错的媒体。这样，如果一个数据包（或数据包的突发）丢失，因为发送前的混合操作，丢包的损失将随着时间推移分摊到多个数据包。它不会导致媒体播放时出现一个很大的缺口。例如，一个包可能含有 220 个立体声采样值，每个都包含一对 16 位数字，通常为 5 毫秒的好品质音乐。如果采样值按顺序发送，一个信息包的丢失表现在音乐流中会出现 5 毫秒的缺失。相反，如果按如图 7-53 所示的方式来发送采样值。所有 10 毫秒时间间隔内的所有偶数采样值都通过一个数据包发送，而所有奇数采样值通过下一个数据包发送。现在，丢失 3 个包的损失并不代表丢失 5 毫秒的音乐，仅仅损失了 10 毫秒内对应的每个其他采样值。媒体播放器很容易地就能处理这种损失：使用前面和后面的样本值交叉插入丢失位。造成的结果是这 10 毫秒的音乐具有较低的临时分辨率，但没有明显的媒体缺失。

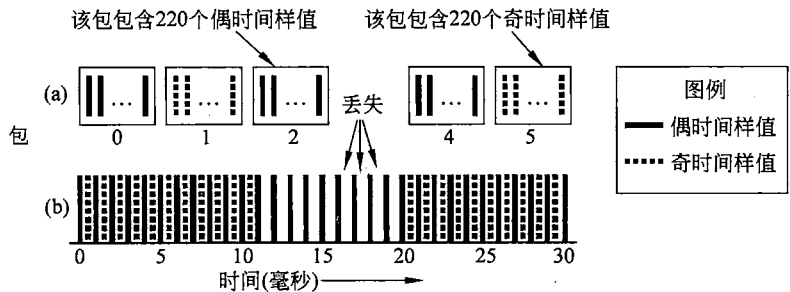


图 7-53

当数据包携带交错的样值时，一个包的丢失只降低了临时分辨率而不会造成播放时间上的沟壑

上面的交错方案只适用于未压缩的采样值。然而,交错发送方式(在短时间内,而不是单个样本值的交替)也可以用于压缩后的样本值,只要有一种方法能找到压缩数据流的采样边界。RFC 3119 给出了一个针对压缩音频的交错方案。

交错编码是一个有吸引力的技术,使用它时无须增加额外的带宽,这点和 FEC 不同;然而,与 FEC 一样,交错编码增加了传输延迟,因为需要等待一组数据包都到达(才能被解交错)。

媒体播放器的第三个工作是解压缩内容。虽然这项任务是计算密集型的,但相当简单。棘手的问题在于如果网络协议不能纠正传输错误,播放器如何解码媒体。在许多压缩方案中,后面的数据不能先于较早的媒体被解压缩,直到较早的数据已经被解压缩出来才能解压缩后面的数据,因为以后的数据是相对于以前数据的编码。对于基于 UDP 的传输协议,还可能出现丢包。因此,编码过程必须设计成不管丢不丢包都能进行解码。这项需求就是为什么 MPEG 采用 I-帧、P-帧和 B-帧的缘故。每个 I-帧可以独立其他帧进行解码,以便从任何先前帧的损失中恢复。

第四个工作是消除抖动,这是所有实时系统的克星。我们在 6.4.3 节描述的一般解决方案是使用播放缓冲区。所有的流媒体系统启动播放前缓冲 5~10 秒的媒体,如图 7-54 所示。播放器定期从缓冲区排出媒体,以便听到清晰的音频和看到平滑的视频。启动延迟给了缓冲区一个机会来填补至低水位标记(low-water mark)。我们的想法是数据现在应该有规律地到达足够多,使得缓冲区永远不会完全被排空。如果缓冲区被排空,那么媒体播放器不得不被卡住。缓冲的价值在于如果有时由于网络拥塞导致数据到达得缓慢,那么缓冲的媒体将保持播放器的正常播放,直至到达新媒体并且缓冲区被补充填满。

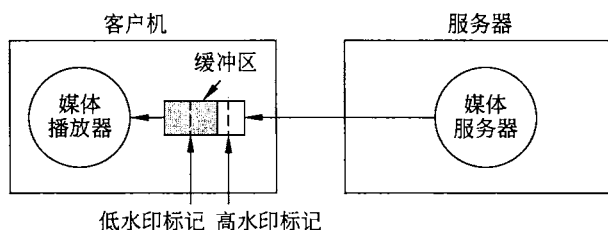


图 7-54 媒体播放器缓冲来自媒体服务器的输入,并且播放缓冲的内容而不是播放直接来自网络的内容

需要多大的缓冲区,媒体服务器要以多快的速率发送媒体才能填满缓冲区,这些决策都取决于网络协议。有很多种可能性。设计中考虑的最大的因素是使用基于 UDP 的传输协议还是使用基于 TCP 的传输协议。

假设采用像 RTP 这种基于 UDP 的传输协议。进一步假设从媒体服务器到媒体播放器有足够的带宽用来发送数据包,只有少量的丢失,而且网络上的其他流量很少。在这种情况下,数据包可按照播放媒体所需要的准确速率发送。每一个数据包通过网络传输,在一定传播延迟后恰好在播放的适当时间到达媒体播放器,然后被播放出来。此时需要的缓冲很少,因为数据包的延迟没有可变性。如果采用交错编码或 FEC 编码,那么需要更多的缓冲,至少需要保证交错或 FEC 得以执行的一组数据包。然而,这仅仅增加了对少量缓冲的需求。

不幸的是,这种场景是不现实的。这里存在两方面的原因。首先,通过网络路径的带

宽变化很大，因此，媒体服务器通常不清楚在它尝试流出媒体之前是否有足够的带宽。一个简单的解决方法是以多种分辨率来编码媒体，并让每个用户选择一个由他 Internet 连接所支持的分辨率。一般来说只有两个层次：高品质的和低质量的，也就是说以 1.5 Mbps 或更高速率的编码能带来高质量的媒体，而以 512 kbps 或更少速率编码的媒体质量较差。

其次，会有一些抖动，或者媒体样值穿越网络所需要的时间会发生变化。这种抖动有两个来源。在网络中通常存在一些相当可观的竞争流量——它们中的某些可能来自多任务用户本身，他们一边浏览网页一边在观看一个流式电影。这种流量会引起媒体到达时间出现波动。而且，我们所关心的是视频帧和音频采样值的到达，而不是数据包的到达。采用压缩技术后，尤其是视频帧可能很大，也可能很小，这完全取决于该帧的内容。一个动作序列通常比一个静止的景观需要更多的比特来编码。因为这些原因，如果网络带宽恒定，媒体的传递随时间的变化率会有所不同。更多的抖动或者延迟的更大变化，要求缓冲区的低水位标记有更大的规模，才能避免欠载造成的播放损失。

现在假设使用基于 TCP 的传输协议来传送媒体，比如 HTTP。因为执行重传机制和等待数据包直到能提供有序的数据包所花费的时间，使得 TCP 增加了媒体播放器可观察到的抖动，而且更加显著。其结果是需要更大的缓冲区和更高的低水位标记。然而，这样做也有一个好处。TCP 将以网络运载数据尽可能快的速度来发送。如果丢失的包必须修复，那么有时候媒体可能会延迟。但大部分时间，网络将能够以快于媒体播放的速度来传递媒体。在此期间，缓冲区将被填满，可起到防止未来欠载的效果。如果网络速度明显快于平均的媒体速率，这是常有的情况，那么启动后不久缓冲区将被迅速填满，这样缓冲区是否会被排空很快将不再是一个令人关切的问题。

基于 TCP 或基于 UDP，并且传输速率超过播放速率，一个问题是媒体播放器和媒体服务器都愿意遥遥领先于播放点开始处理媒体。它们往往愿意下载整个文件。

但是，遥遥领先于播放点开始工作其实是不必要的，这样可能需要大量的存储空间，对避免缓冲区欠载并没有必要。当不想要太早开始处理时，媒体播放器的解决方案是定义一个缓冲区的高水位标记（high-water mark）。基本原则是服务器不断地输出数据直到缓冲区被填满至高水位标记。然后媒体播放器告诉服务器暂停发送。由于数据将继续源源不断地流来，直到服务器收到暂停请求，缓冲区中高水位标记和缓冲区末尾之间的距离必须大于网络的带宽延迟乘积。在服务器停止后，缓冲区将开始排出。当它降低到低水位标记时，媒体播放器告诉媒体服务器重新启动发送。为了避免欠载，在要求媒体服务器恢复发送媒体时，低水位标记必须考虑网络的带宽延迟乘积。

要启动和停止媒体流，媒体播放器需要对其进行远程控制。这就是 RTSP 协议所提供的功能。该协议在 RFC 2326 中被定义，它给播放器提供了控制服务器的机制。除了启动和停止媒体流外，它可以后退或前进到某个位置、播放特定的时间间隔以及快进或者快退。它不提供数据流的传输，不过，这通常是 UDP 之上的 RTP，或者基于 TCP 的 HTTP 之上的 RTP 的任务。

图 7-55 列出了 RTSP 协议所提供的主要命令。这些命令有一个简单的文本格式，类似于 HTTP 消息，并且通常通过 TCP 来传递命令。RTSP 也可以在 UDP 上运行，因为每个命令都需要确认（并且，如果没有获得确认，就必须重新发送）。

命令	服务器动作
DESCRIBE	列出媒体参数
SETUPE	在播放器和服务器之间建立一个逻辑信道
PLAY	开始给客户发送数据
RECORD	客户开始接受数据
PAUSE	暂停发送数据
TEARDOWN	释放逻辑信道

图 7-55 播放器发给服务器的 RTSP 命令

尽管 TCP 似乎不太适合实时流量的传输，但在实际上它还是经常被采用。最主要的原因是它比 UDP 更容易穿过防火墙，尤其是在通过 HTTP 端口运行时。大多数管理员把防火墙配置成保护他们的网络被不受欢迎的访客访问。他们几乎总是允许来自远程端口 80 的 TCP 连接，以便 HTTP 和 Web 流量通过，因为阻塞该端口将迅速引发校园网的不满。然而，大多数其他端口还是被封锁的，其中包括 RSTP 和 RTP，它们分别使用端口 554 和端口 5004。因此，流媒体通过防火墙的最简单的方式是假装它是一个 HTTP 服务器，正在发送一个普通的 HTTP 响应，至少要让防火墙这么认为。

使用 TCP 还有其他一些优点。因为它提供了可靠性，TCP 给客户提供的媒体是一个完整副本。这很容易让用户倒回到某个播放点观看而不用担心数据丢失。最后，TCP 将尽可能快地缓冲尽可能多的媒体。当缓冲空间变得越来越便宜（当磁盘用于存储时），媒体播放器可以在用户观看的同时下载媒体。一旦下载完成，用户就可以不间断地观看，即使他丢失网络连接也无妨。这个属性对移动电话非常有用，因为它们而言在运动中与网络的连接可能快速变化着。

TCP 的缺点是增加了启动延迟（因为 TCP 启动需要时间），同时也增加了更高的低水位标记。然而，只要网络带宽超过媒体速率相当多个量级，这点只是很少的一个惩罚。

7.4.4 流式直播媒体

不仅只有录制的视频在网络上红极一时。直播流媒体也非常受欢迎。一旦在 Internet 上观看音频和视频流成为可能，商业电台和电视台就都有了通过 Internet 以及空中广播其节目的想法。没过多久，学院广播站也开始把它们的信号放在 Internet 上。然后，大学生开始了他们自己的 Internet 广播。

今天，人们和各种规模的企业通过网络发送实况音频和视频。这个领域是技术和标准演变创新温床。直播流媒体用于各大电视台的网上业务，这就是所谓的 IP 电视（IPTV，IP TeleVision）。它也可以用作广播电台，例如 BBC，这就是所谓的网络广播（Internet radio）。IPTV 和 Internet 广播向全球观众报道各类事件，从时装表演到世界杯足球赛，再到墨尔本板球场测试赛的直播。通过 IP 直播媒体流作为一种技术还被有线电视运营商用来建立自己的广播系统。而且它已被广泛用于低预算的运营，从成人网站到动物园网站。使用目前的技术，几乎任何人都可以快速启动流媒体直播，并且所需费用很少。

流媒体直播的方法之一是把节目记录到磁盘上。观众可以连接到服务器的存档文件，拖拽任何节目，并下载下来慢慢听。播客（podcast）就是以这种方式检索的一个片段。对于预先设定的事件，也可以在现场直播之后把内容存储起来，因此存档是正在进行的，或

者说，半小时或更少时间后的直播内容。

事实上，这种做法和我们刚刚讨论过流式媒体所使用的相同。这是容易做到的，我们讨论过的所有技术都可以完成它的工作，而且观众可以从存档文件中的所有节目中挑选自己喜欢的。

另一种不同的方法是在 Internet 上广播实况。观众们调到正在直播的媒体流，就像打开电视机一样。然而，媒体播放器为用户提供的附加功能更多，用户可以暂停或倒转回放媒体。实况媒体将继续流向播放器，被播放器缓冲起来直到用户已经准备好观看。站在浏览器的角度，它看起来就像流存储媒体一样。对于播放器而言，内容是否来自一个文件或者是通过网络正在发送过来的直播根本不重要，而且通常也不告诉他（除了不可能往前快进一个实况流之外）。由于机制的相似性，我们前面讨论的许多内容都适用，但也有一些关键的差别。

重要的是，仍然需要在客户端进行缓冲，以便平滑抖动。事实上，对于流媒体直播通常需要大量的缓冲（独立于用户可能暂停播放作做的考虑）。当媒体流来自一个文件，那么可以一个大于播放速率的速率把媒体推送出来。这种做法将迅速建立一个缓冲区，来补偿网络抖动（如果它不希望缓冲更多的数据，那么播放器将停止流）。相比之下，实况流媒体总是以它产生的精确速率传输，这个速率也是与它被播放的速度相同。它不能以更快的速率发送。因此，缓冲区必须大到足以处理全方位的网络抖动。实际上，10~15 秒的启动延迟通常就足够了，所以这不是一个大问题。

其他重要的区别在于流媒体直播事件通常拥有数百或数千个同时观看相同内容的观众。在这种场景下，流媒体直播很自然的解决方案是使用组播。这与流存储媒体的情况不同，因为那种情况下的用户在任何特定时间通常观看不同的内容。于是，直播给许多用户同时观看的流由许多单个的流会话组成，这些会话发生在同一时间。

组播流媒体的方案以如下方式工作。服务器将每个媒体包通过 IP 组播一次发给一个组地址。网络给每个组成员传递一份包的副本。要想接收媒体流的客户必须加入该组。客户使用 IGMP 就可做到这一点，而不必向媒体服务器发送一个 RTSP 消息。这是因为媒体服务器早就已经在发送实时流了（之前第一个用户加入情况除外）。我们所需要的就是安排在本本地接收流。

由于组播是一个一对多的传递服务，媒体被 RTP 包携带，并通过 UDP 传输。TCP 仅用在单个发送者和单个接收者之间。由于 UDP 不能提供可靠性，可能会丢失一些数据包。为了把媒体丢失水平降低到可接受的水平范围内，我们可以使用前面所述的 FEC 和交错编码技术。

在 FEC 的情况下，存在一个组播的良性互动，如图 7-56 所示的奇偶实例。当数据包被组播发送时，不同的客户可能会丢失不同的数据包。例如，客户端 1 丢失了数据包 B，客户端 2 丢失了奇偶包 P，客户端 3 丢失了数据包 D，而客户端 4 没有丢失任何数据包。然而，即使在客户端丢失了 3 个不同的数据包，这个例子中的每个客户端仍然可以恢复所有的数据包。所有这一切的前提是每个客户端不能丢失超过一个包，不论丢哪一个都无所谓，因此丢失的数据包可以通过奇偶计算得以恢复。（Nonnenmacher 等，1997）描述了如何利用这个想法来提高组播传输的可靠性。

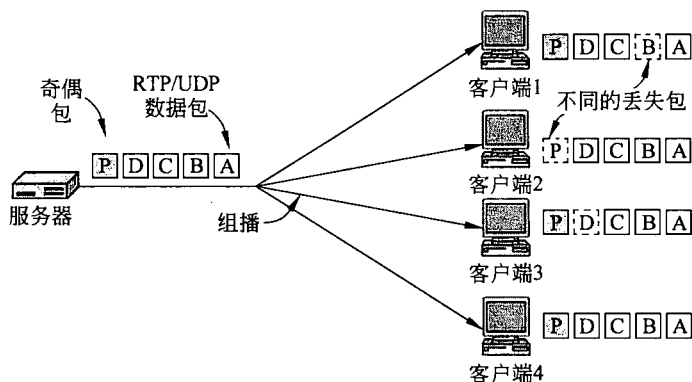


图 7-56 带有奇偶包的组播流媒体

对于拥有大量客户的服务器来说，以 RTP 和 UDP 数据包来组播流媒体无疑是最有效的运营方式。否则，服务器有 N 个客户时必须发送 N 个流，对于同时需要处理大量流事件的服务器来说将需要非常大的网络带宽。

实际上 Internet 不是这样工作的，看到这里你可能会大吃一惊。通常发生的情况是每个用户与服务器建立一个单独的 TCP 连接，媒体流通过该连接传给用户。对客户端来说，这与观看流存储媒体的方式是相同。至于流存储媒体，之所以有这个看似糟糕的选择存在以下几个原因。

第一个原因是 IP 组播在 Internet 上并没有真正获得部署。一些 ISP 和网络仅在自己内部支持它，但它通常不支持跨越其网络边界的组播，因为这需要广域的流传输。其他原因还在于 TCP 相比 UDP 具有某些优势，正如前面所讨论的那样。基于 TCP 传输的媒体流几乎能到达 Internet 上所有的客户，尤其是当伪装成 HTTP 后可穿越防火墙，而且可靠的媒体流传递允许用户执行倒带回放。

然而，在一个重要情况下可以用 UDP 来组播媒体流：在服务提供商的网络内部。例如，有线电视公司可能决定给客户机顶盒广播电视频道，具体做法是使用 IP 技术来代替传统的视频广播。利用 IP 包分发广播视频的技术被广泛地称为 IPTV，如上所述。由于有线电视公司可以完全控制自己的网络，所以它可以把自己的网络工程化为支持 IP 组播，而且为基于 UDP 的分发分配足够的带宽。所有的这一切对于客户都是不可见的，因为 IP 技术只存在于供应商的围墙花园（walled garden）内。在服务方面它看起来就像有线电视，但是它的基础是 IP，机顶盒是运行 UDP 的计算机，电视机只是简单连接到计算机的显示器。

返回到 Internet 的情形，在 TCP 上面直播流媒体的缺点是服务器必须为每个客户端发送一个单独的媒体副本。这对于用户中等数量的客户群是可行的，特别是传输音频流。这里的诀窍在于找到一个具有良好 Internet 连接的位置放置媒体服务器，使得它有足够的带宽。通常，这意味着向托管服务提供商租用一台数据中心的服务器，而不是使用家里一台只有宽带 Internet 连接的服务器。托管服务提供商的市场竞争力非常激烈，因此租用一台服务器不会很昂贵。

事实上，任何人，甚至一个学生，都很容易建立和运营一个流媒体服务器，比如一个 Internet 电台。这个电台的主要组成部分如图 7-57 所示。电台的基础是一台普通 PC，带有一个像样的声卡和麦克风。运行流行的软件来捕获音频，并将之编码成各种格式，例如

MP4，媒体播放器像往常一样用来收听音频。

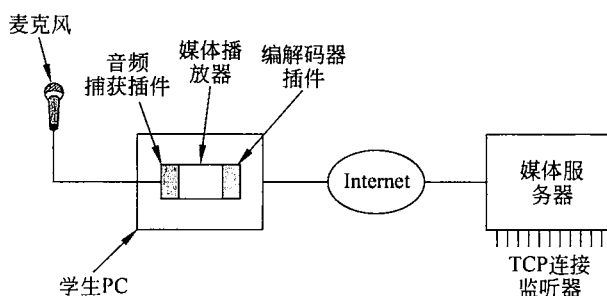


图 7-57 一个学生广播电台

然后，在 PC 上捕获的音频流通过 Internet 被送入一个媒体服务器，作为播客的存储文件流或直播媒体流。该媒体服务器具有良好的网络连接，它处理大量通过 TCP 连接分发媒体的任务。同时该服务器还承担前端 Web 站点的角色，提供电台的介绍页面以及指向可用流媒体内容的链接。所有这些事情都可利用商业软件包来负责管理，也可利用开源软件包来实施，比如 icecast。

然而，针对一个拥有非常大量客户群的电台来说，一台服务器使用 TCP 将流媒体发送到每个客户端的做法就变得不可行了。简单地说，一台服务器根本没有足够的带宽来做这件事情。对于大型流媒体网站，流媒体的分发使用了一组服务器来完成的，而且这组服务器在地理上是分散的，从而使客户端可以连接到最近的服务器。这就是内容分发网络，我们将在本章结尾处学习这部分内容。

7.4.5 实时会议

曾几何时，在公共交换电话网络上可以进行语音通话，而且网络流量主要是语音流量，数据流量只在这里和那里存在一点点。然后，Internet 来了，Web 出现了。数据流量增长、增长、再增长，直到 1999 年数据流量和语音流量一样多（因为语音现在都被数字化了，所以语音流量和数据流量都可以测量比特数）。到 2002 年，数据流量超过语音流量一个量级，并且仍然呈几何级数增长趋势，反观语音流量基本保持持平。

这种增长的后果彻底翻转了电话网络。现在语音流量也用 Internet 技术来运载了，而且只占一小部分的网络带宽。这种颠覆性的技术称为 IP 语音（voice over IP），也称为 Internet 电话（Internet telephony）。

IP 语音的使用形式各种各样，基本上都是由强大的经济因素所驱动（它能节省金钱，因此人们使用它）。一种形式看起来像普通电话（老式电话？）一样，但是电话插入以太网，并通过网络发出呼叫。Pehr Anderson 是麻省理工学院的本科学生，当他和他的朋友为一个课程项目原型实现了这个设计，结果他们得到的成绩是“B”。愤愤不平的他在 1996 年创办了一个称为 NBX 的公司，并开创了这种基于 IP 的语音传输；3 年后，他将公司以 9000 万美元出售给 3Com。公司喜欢这种方法，因为有了这种技术可以让他们摆脱单独的电话线，并且用他们已经拥有的网络来做这件事。

另一种方法是利用 IP 技术来建立一个长途电话网络。在一些国家，比如美国，这个网

络可以用来接入有竞争力的长途电话服务。用户只需要拨打一个特殊的号码前缀，然后包含其声音采样值的数据包被注入到网络，并通过网络传输；当这些数据包离开网络时其中的声音采样值被提取出来。由于 IP 设备比电信设备便宜得多，因此这导致了更廉价的服务。

顺便说一句，在价格上的差异并不完全是技术性的。几十年来，电话服务是一个受管制的垄断企业，保证电话公司在其成本之上有一个固定百分比的利润。毫不奇怪，这样导致了成本的上涨，例如，有很多很多的冗余硬件只是以提供更好的可靠性为名义（电话系统只允许每 40 年故障 2 个小时，平均 3 分钟一年）。这种效果通常称为“镀金电线杆综合征”。由于放松了管制，效果已明显下降，当然，传统设备仍然存在。IT 行业从未有过任何类似的经营历史，所以它总在不断地精益求精。

不过，我们将专注于 IP 语音的形式，这可能对用户是最直观的：使用一台计算机呼叫另一台计算机。这种形式伴随着个人计算机开始与麦克风、扬声器、相机捆绑销售、CPU 速度足够快到能处理媒体，以及人们开始在家里以宽带速率连接到 Internet 等一系列的软硬件环境变化逐步普及的。一个众所周知的例子是 2003 年发布的 Skype 软件。Skype 和其他公司还提供网关，使得用户很容易地呼叫固定电话号码，同时也能呼叫计算机的 IP 地址。

随着网络带宽的增加，视频电话加入到了语音通话中。最初，视频通话主要在公司内部使用。视频会议系统设计成能交换两个或两个以上位置的视频，使得不同地点的管理人员在举行会议时能够看到对方。然而，有了良好的宽带 Internet 连接和视频压缩软件，家庭用户也可以参与视频会议。诸如 Skype 这样的工具，最初只能路由音频，现在能够路由视频，因此分布在世界各地的朋友和家人不仅可以看到对方还能听到对方。

从我们的角度来看，Internet 语音或视频通话也属于流媒体问题，但是比存储的流媒体文件或现场直播活动的限制更大。这里新增的约束在于双向通话所必需的低延迟。电话网络允许单向延迟至多为 150 毫秒，这是用户可接受的最低限度，超过这个延迟就会被与会者视为烦人而拒绝（国际长途电话可能有高达 400 毫秒的延迟，这一点严重损害了用户体验的积极性）。

这样的低延迟实际上很难达到。当然，缓冲 5~10 秒的媒体是无法工作（就像它广播一场体育赛事实况）。相反，视频和 IP 语音系统必须设计并实现多种技术来最小化延迟。这一目标的明确意味着选择 UDP 而不是 TCP，因为 TCP 的重传机制至少引入了一个往返延迟。然而，一些形式的延迟是不能减少，即使使用 UDP 也是如此。例如，西雅图和阿姆斯特丹之间的距离接近 8000 千米，在这么长距离的光纤上光的传播延迟为 40 毫秒。好啦，祝你好运。实际上，通过网络的传播延迟时间会更长，因为它要覆盖一个较大的距离（比特不会遵循一个大的循环路由），而且还有每个 IP 路由器存储和转发数据包的传输延迟。这种固定的延迟时间消耗了可接受的延迟预算。

延迟的另一个根源与数据包大小有关。通常情况下，大的包是使用网络带宽的最好方式，因为它们更高效。然而，一个 64 kbps 的音频采样率，1 KB 的数据包需要 125 毫秒来填补（甚至更长的时间，如果采样值被压缩）。这种延迟会消耗大部分的总延迟预算。此外，1 KB 的数据包如果通过 1 Mbps 的宽带接入链路发送将需要 8 毫秒的传输时间。然后再加上另一端宽带连接所需要的另一个 8 毫秒。显然，大的数据包将无法工作。

相反，IP 语音系统使用短的数据包，以牺牲带宽效率来换取延迟的降低。它们打包成一批更小单位的音频样本，通常 20 毫秒一个。按照 64 kbps 速率，数据长 160 字节，有少

量压缩。然而，根据定义，这个数据包的延迟将是 20 毫秒。传输延迟也会小一些，因为包的长度很短。在我们的例子中，它会减少到大约 1 毫秒。通过使用短的数据包，西雅图到阿姆斯特丹的最低单向延迟已经从一个不可接受的 181 毫秒（ $40+125+16$ ）减少到可以接受的 62 毫秒（ $40+20+2$ ）。

我们甚至还没有谈论软件开销，但是它也会吃掉一些延迟预算。尤其对于视频的确如此，因为通常需要对视频进行压缩才能使得视频适应当前的可用带宽。这和一个存储的流媒体文件传输完全不同，此时没有时间运行一个计算密集的编码器来获得高层次的压缩比，编码器和解码器都必须快速地运行。

缓冲仍然需要，主要作用还是用来按时播放媒体样值（以避免难以理解的音频和忽停忽动的视频），但是缓冲量必须保持得非常小，因为在我们延迟预算中的剩余时间是以毫秒为单位计量的。当一个数据包经过很长时间到达后，播放器必须跳过丢失的样本，也许用环境噪声或重复帧给用户掩盖损失的那部分媒体。这里用于处理抖动的缓冲区大小和丢失的媒体量之间存在着一个权衡。较小的缓冲区能减少延迟，但会导致由于抖动带来的更多丢失。最终，随着缓冲区空间的缩小，对用户而言丢失将变得更为明显。

细心的读者可能已经注意到在本节到目前为止的讨论中，我们没有提及任何有关网络层协议的内容。通过使用服务质量机制，网络可以减少延迟，或者至少减少抖动。这个问题之所以一直没有提出来的原因在于，以前尽管有很大的延迟流媒体还是能够运作，即使在实况流的情况下。如果延迟不是一个大问题，那么终端主机的缓冲区就足以应付抖动问题。然而，对于实时会议系统，通常减少网络延迟和抖动非常重要，这样才有助于媒体流满足延迟预算。唯一不重要的是当有这么多个网络带宽可用时，每个人都能得到良好的服务。

第 5 章，我们描述了有助于实现这个目标的两个服务质量机制。一种机制是区分服务（DS, Differentiated Services），在这种机制下，包被标记为不同的类别，在网络内部将受到不同的处理。适当的做法是将语音 IP 数据包标记为低延迟。实际上，系统将 DS 编码点设置成众所周知的加速转发类（Expedited Forwarding）和低延迟型服务（Low Delay）。这在宽带接入链路特别有用，因为这些链路往往拥挤不堪，当 Web 流量或其他流量交织在一起竞争使用链接时。对于一条稳定的网络路径，拥塞会造成延迟和抖动的增加。每 1 KB 的数据包在 1 Mbps 的链路上发送需要 8 毫秒，而且被排在 Web 流量的后面，那么语音 IP 数据包将遭受拥塞带来的延误。然而，打上低延迟标记的语音 IP 数据包将会跳到队列的前头，绕过 Web 数据包，由此来降低延迟。

可以用来减少延迟的第二个机制是确保有足够的带宽。如果可用带宽变化不定，或者传输速率波动不稳（犹如压缩视频），并且有时没有足够的带宽，那么在路由器上将建立队列，并且随着队列的增长而延迟增加。这种情况甚至在 DS 机制下也会发生。为了确保有足够的带宽，在网络内部可以采用预留资源的方法。这种能力就是综合服务所提供的。非常遗憾的是这种机制并没有得到广泛部署。相反，网络被工程化为预期的流量级别，或者网络客户获得一份针对给定流量级别的服务等级约定书。应用程序必须低于这个级别运行，以免出现网络拥塞并造成不必要的延误。对于在家偶尔使用的视频会议，用户可以选择一个视频质量作为带宽需求的代理，或者由该软件测试网络路径，并自动选择适当的质量。

上述任何一个因素都可导致数据包的延迟成为不可接受，因此，实时会议要求应该高度重视所有这些因素。有关 IP 语音概述和影响延迟的因素分析报告，请参阅（Goode, 2002）。

既然我们已经讨论了媒体流路径上的延迟问题，我们现在将注意力移动到会议系统必须解决的其他主要问题。这个问题就是如何建立和拆除呼叫。我们将着眼于两个被广泛应用的协议，H.323 和 SIP。Skype 是另一个重要的系统，但其内部工作原理是私有的。

H.323

在开始通过 Internet 进行音频和视频呼叫之前，有一件事情是每个人都很清楚的，那就是如果每个厂商都设计它自己的协议栈，那么系统将无法工作。为了避免这个问题，许多感兴趣的团体在 ITU 的倡导下，聚集在一起制定相关标准。1996 年，ITU 发布了推荐标准 **H.323**，其标题是“无服务质量保障的局域网可视电话系统与设备”（Visual Telephone Systems and Equipment for Local Area Networks Which Provide a Non-Guaranteed Quality of Service）。只有电话行业才会想得出这样一个名字。很快在 1998 年它就被修订成“基于数据包的多媒体通信系统”。H.323 是第一代应用广泛的 Internet 电话系统的基础，它仍然是获得最广泛部署的解决方案，2009 年发布了第 7 个版本。

与其说 H.323 是一个特定的协议，不如说它是 Internet 电话总体结构的概述。在语音编码、呼叫建立、信令、数据传输和其他领域中，H.323 引用了大量特殊的协议，而不是自己制定一套新规定。图 7-58 描述了它的总体模型。在中心处是一个网关（gateway），它将 Internet 与电话网络连接起来。网关在 Internet 一端使用 H.323 协议，在电话端则使用 PSTN 协议。通信设备称为终端（terminal）。一个 LAN 可能有一个网守（gatekeeper），它控制其管辖范围内的端点，这个管辖范围称为区域（zone）。

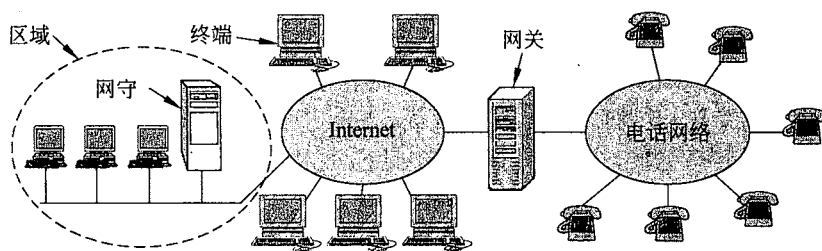


图 7-58 Internet 电话的 H.323 体系结构模型

电话网络需要许多协议。首先，需要一个协议用来对音频和视频进行编码和解码。标准电话把一个语音信道表示成 64 kbps 的数字语音（每秒 8000 次采样），这是由 ITU 推荐的标准 **G.711** 所定义的。所有的 H.323 系统必须支持 G.711。其他压缩语音的编码方法也是允许的，但不是必需的。这些方法采用了不同的压缩算法，并且在质量和带宽之间作出不同的权衡。对于视频，必须支持我们前面描述过的视频压缩 MPEG 形式，包括 H.264。

由于允许使用多个压缩算法，所以为了允许终端协商即将使用哪一个算法，还需要一个协议。这个协议称为 **H.245**。它还能协商一个连接的其他方面，比如数据速率。RTCP 被用来控制 RTP 信道。还需要有一个协议用来建立和释放连接、提供拨号音、产生响铃声以及完成标准电话的其他功能。这里使用的是 **ITU Q.931**。终端需要一个协议跟网守（如果有的话）进行通信，**H.225** 可作为此用途。它管理的从 PC 到网关的信道称为“注册/许可/状态”（RAS, Registration /Admission /Status）信道。该信道允许终端加入和离开该区域、请求和返回带宽、提供状态更新，以及其他一些事情。最后，还需要有一个协议用于实际

的数据传输，UDP 之上的 RTP 可用作此用途。如同往常一样它由 RTCP 来管理。所有这些协议的相互关系如图 7-59 所示。

音频	视频	控制			
G.7××	H.26×	RTCP	H.225 (RAS)	Q.931 (信令)	H.245 (呼叫控制)
RTP					
UDP				TCP	
IP					
链路层协议					
物理层协议					

图 7-59 H.323 协议栈

为了看清楚这些协议是如何相互配合工作的，考虑在一个 LAN（有一个网守）中有一台 PC 终端需要呼叫一部远程电话的情形。该 PC 首先必须找到网守，因此它向端口 1718 广播一个 UDP 网守发现包；当网守有了回应以后，PC 就知道了网守的 IP 地址。现在，PC 向网守发送一条 RAS 注册消息，该消息被封装在一个 UDP 数据包中。在该消息被接受以后，PC 向网守发送一条 RAS 许可消息，请求所需要的带宽。只有在获得了所需的带宽以后，PC 才可以开始请求建立呼叫。事先请求带宽的做法是为了让网守限制呼叫的数量，避免超额使用出境线路，从而有助于提供必要的服务质量。

顺便说一句，电话系统做了同样的事情。当你拿起听筒，一个信号就被发送到本地端局。如果本地端局有足够的空闲容量用作一个呼叫，它会产生一个拨号音。如果没有，你将什么也听不到。如今，电话系统的规模已经超大到你几乎瞬间就能听到拨号音，但在初期的电话中，往往要花几秒钟才能听到拨号音。因此，如果你的孙子辈问你“为什么有拨号音？”现在你知道答案了。除非到那时，电话已不再存在。

为了建立呼叫，现在 PC 建立了一个到网守的 TCP 连接。建立呼叫使用了现有的电话网络协议，这些协议都是面向连接的，因此需要使用 TCP。相反地，电话系统不需要电话机来宣布自己的存在，因此没有像 RAS 这样协议，所以 H.323 的设计者可以自由使用 UDP 或 TCP 来实现 RAS，最终他们选择了开销较小的 UDP。

既然该 PC 已经被分配了所需要的带宽，它就可以通过 TCP 连接发送一条 Q.931 SETUP 消息。该消息指定了被叫电话的号码（如果被呼叫的是一台计算机，则给出的是 IP 地址和端口）。网守以一条 Q.931 CALL PROCEEDING 消息作为响应，确认它已正确地收到了请求。然后网守将 SETUP 消息转发给网关。

网关的一半是计算机，另一半是电话交换机，它向目标（普通）电话发出一个普通电话呼叫。目标电话所在的端局使被叫电话发出响铃声，同时该端局还发回一条 Q.931 ALERT 消息，告诉主叫方 PC 电话已经开始响铃了。当另一端的用户拿起电话时，该端局发送回一条 Q.931 CONNECT 消息，以便通知 PC 呼叫连接建立好了。

一旦连接被建立，网守就不再出现在通信过程中，当然网关仍然还在。后续的数据包将绕过网守，直接抵达网关的 IP 地址。至此，我们才有了一个运行于两端之间的直接管道。这只是一个用来传输比特的物理层连接，仅此而已没有其他。通信双方中的任何一方对另

一方都一无所知。

现在使用 H.245 协议来协商当前呼叫的参数, 这里使用的是 H.245 控制信道, 它始终是打开的。每一方都从宣布它的处理能力开始, 例如, 它是否能处理视频 (H.323 可以处理视频) 或会议呼叫、它支持哪些编解码器, 等等。一旦一方知道了另一方具备哪些处理能力以后, 就建立两个单向的数据信道并且为每个信道指定一个编解码器和其他一些参数。由于每一方可能有不同的设备, 所以前向和逆向信道上的编解码器有可能完全不同。当所有的协商工作都完成以后, 就可以开始用 RTP 来传送数据流了。RTP 的管理由 RTCP 负责, RTCP 在拥塞控制中起着重要的作用。如果有视频的话, RTCP 还要处理音频/视频的同步。图 7-60 显示了各种信道。当任何一方挂断电话时, 为了释放不再需要的资源, 在完成通话后使用 Q.931 呼叫信令信道来拆除连接。

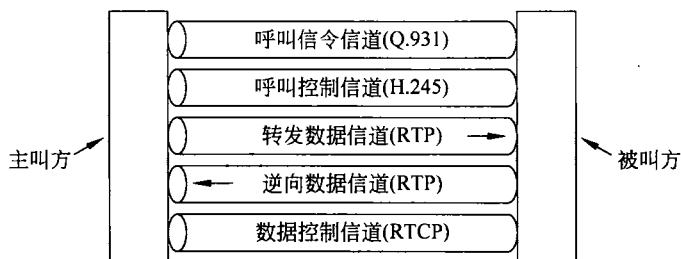


图 7-60 一次呼叫过程中主叫和被叫之间的逻辑信道

当呼叫结束时, 主叫方 PC 再次用一条 RAS 消息与网守联系, 以便释放分配给它的带宽。或者, 它也可以启动另一次呼叫。

我们没有涉及 H.323 中有关服务质量的任何内容, 即使我们说过这是实时会议取得成功的一个非常重要部分。之所以没有讨论这部分内容是因为 QoS 超出了 H.323 的范围。如果底层网络能够提供一个从主叫 PC 到网关之间稳定而无抖动的连接, 那么该呼叫的服务质量就会很好; 否则, 服务质量无法得到保证。然而, 电话那端的呼叫部分将是无抖动的, 因为电话网络就是这样设计的。

SIP——会话发起协议

H.323 是由 ITU 设计的, Internet 社团中的许多人把它看成是一个典型的电信产品: 庞大、复杂而且不灵活。因此, IETF 建立了一个委员会, 专门设计一种更加简单和模块化的方法来实现 IP 语音。迄今为止的主要成果是会话发起协议 (SIP, Session Initiation Protocol), 最新版本是 RFC 3261, 发布于 2002 年。该协议描述了如何建立 Internet 电话呼叫、视频会议和其他多媒体连接。H.323 是一个完整的协议集, 与此不同的是 SIP 只是单个模块, 但是它被设计成能很好地与现有的 Internet 应用协同工作。例如, 它将电话号码定义成 URL, 所以 Web 页面可以包含电话号码, 用户只要单击一个链接就可以发起一次电话呼叫 (与 mailto 方法相同, mailto 允许用户单击一个链接来启动发送电子邮件消息的程序)。

SIP 可以建立两方会话 (普通的电话呼叫)、多方会话 (每个人都可以听和说) 以及组播会话 (一个发送方, 多个接收方)。这些会话可以包含音频、视频或数据, 后者对于诸如多方实时游戏之类的应用非常有用。SIP 只处理会话的建立、管理和终止。还需要使用诸如 RTP/RTCP 之类的其他协议来实施数据传输。SIP 是一个应用层协议, 它可以运行在 UDP

或 TCP 之上。

SIP 支持多种服务，包括查找被叫方的位置（他可能不在家里的计算机旁边）、确定被叫方的处理能力，以及处理呼叫建立和终止的机制。在最简单的情形中，SIP 建立一个从主叫方计算机到被叫方计算机之间的会话，所以我们首先来讨论这种情形。

SIP 中的电话号码被表示成采用 sip 方案的 URL，例如，sip:ilse@cs.university.edu 代表一个 DNS 域名为 cs.university.edu 主机上一个名叫 ilse 的用户。SIP URL 也可以包含 IPv4 地址、IPv6 地址或者实际的电话号码。

SIP 协议是一个仿照 HTTP 的基于文本的协议。一方以 ASCII 文本的形式发送一条消息，消息的第一行包含一个方法名，接下来的几行包含一些传递参数的头。很多头都来自于 MIME，以便 SIP 能与现有的 Internet 应用协同工作。图 7-61 列出了核心规范中定义的 6 种方法。

方法	描述
INVITE	请求发起一次会话
ACK	确认会话已经启动
BYE	请求终止会话
OPTIONS	查询主机的能力
CANCEL	取消正在进行中的请求
REGISTER	通知服务器重定向用户的当前位置

图 7-61 SIP 方法

为了建立一个会话，主叫方要么与被叫方建立一个 TCP 连接并在其上发送一条 INVITE 消息，要么以 UDP 数据包形式发送 INVITE 消息。在这两种情况下，第二行和随后各行中的头描述了消息体的结构，其中包含主叫方的处理能力、媒体类型和格式。如果被叫方接受此次呼叫，那么它回应一个 HTTP 类型的应答码（使用图 7-48 中响应组的三位数字，200 表示接受）。在应答码行的后面，被叫方还可以提供有关自己的处理能力、媒体类型和格式等信息。

利用三次握手方法，可以建立一个连接，所以主叫方以一条 ACK 消息作为响应，以便结束此协议，并确认已收到了 200 消息。

双方中的任何一方都可以通过发送一条包含 BYE 方法的消息来结束当前会话。当另一方确认了该消息，会话就结束了。

OPTIONS 方法被用于向一台机器查询它的处理能力。通常主叫方在发起一个会话前使用 OPTIONS 方法来查明该机器是否具备处理 IP 语音的能力，或者预期的会话类型。

REGISTER 方法与 SIP 的跟踪能力有关，SIP 能够追踪一个用户，并且当该用户不在家时与他建立连接。该消息被发送给一个 SIP 位置服务器，正是该位置服务器记录了每个用户当前在哪里。以后，可以向它查询某个用户的当前位置。重定向的操作如图 7-62 所示。在这里，主叫方将 INVITE 消息发送给一个代理服务器，以便把可能的重定向过程隐藏起来；然后代理服务器查找被叫用户的位置信息，并将 INVITE 消息发送至该处；然后，代理服务器为后续的三次握手过程中的消息充当中继节点。LOOKUP 和 REPLY 消息并不是 SIP 的一部分；这里可以使用任何方便的协议，取决于使用了什么类型的位置服务器。

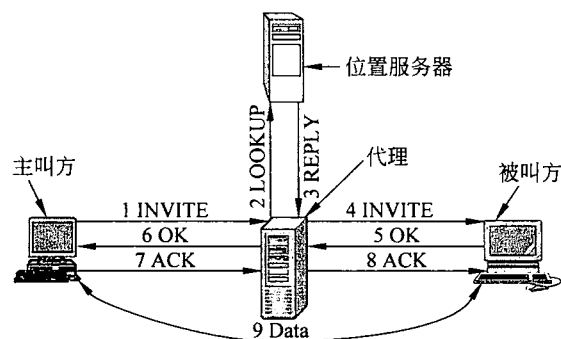


图 7-62 使用代理服务器和 SIP 重定向

SIP 还有许多其他功能，其中包括呼叫等待、呼叫屏蔽、加密和认证等，我们这里不再一一赘述。如果在 Internet 和电话系统之间有可用的合适网关，则 SIP 还具备从计算机向普通电话发起呼叫的能力。

H.323 与 SIP 比较

H.323 与 SIP 都允许用计算机和电话作为端点来实现两方和多方呼叫。它们都支持参数协商、加密和 RTP/RTCP 协议。图 7-63 总结了它们之间的相似之处和不同之处。

项目	H.323	SIP
由谁设计	ITU	IETF
是否与PSTN兼容	是	大部分是
是否与Internet兼容	是，随时间推移	是
体系结构	庞大	模块化
是否完整	完整的协议栈	只有用来控制建立的SIP
是否允许参数协商	是	是
呼叫信令	TCP之上的Q.931	TCP/UDP之上的SIP
消息格式	二进制	ASCII
媒体传输	RTP/RTCP	RTP/RTCP
是否支持多方呼叫	是	是
多媒体会议	是	否
寻址方式	URL或电话号码	URL
呼叫终止	显式或TCP释放	显式或超时
即时消息	否	是
加密	是	是
标准大小	1400页	250页
实现	庞大而复杂	中等，但有问题
状态	世界范围，尤其视频	可替代，尤其音频

图 7-63 H.323 和 SIP 的对比

尽管两个协议的功能集类似，但它们的哲学思想却相差甚远。H.323 是一个典型的重量级电话工业标准，它规定了完整的协议栈，并且精确定义了什么是允许的，什么是禁止的。这种方法使得每一层都有定义完好的协议，从而使得互操作性的任务变得容易。但是，其付出的代价是一个庞大的、复杂的、严格的标准，而且这个标准难以适应未来的应用。

与此相反，SIP 是一个典型的 Internet 协议，只需要交换短短几行 ASCII 文本就可以工作。它是一个轻量级的模块，与其他的 Internet 协议协同工作得很好，但是与现有电话系

统的信令协议一起工作则稍差一些。

因为 IETF 的 IP 语音模型是高度模块化的，所以它很灵活，并且很容易适应新的应用。其缺点是遭遇的如何实现互操作性，尤其当人们试图解释什么是标准化方法时比 H.323 稍逊一筹。

7.5 内容分发

Internet 通常被用于通信目的，就像电话网络一样。在早期，研究者要与远程计算机通信，必须通过网络登录到远程机器才能执行任务。很长一段时间人们使用电子邮件来互相沟通，现在，大家都使用视频和 IP 语音。然而，自从 Web 出现之后，在 Internet 上内容已经多于通信。很多人使用网络查找资料，受到电影、音乐和节目访问的驱动，出现了大量可共享的对等文件。内容交换已如此明显，以至于现在大多数 Internet 带宽被用来传递存储的视频。

由于分发内容的任务与通信完全不同，它对网络提出了不同的需求。例如，如果 Sally 想与 Jitu 谈话，她可能向他的移动电话发出 IP 语音呼叫。通信必须在特定的计算机上完成；它呼叫 Paul 的计算机将不能做得很好。但是，如果 Jitu 想观看他团队的最新板球比赛，他很幸福地可以从任何提供服务的计算机上获得视频流。他不介意计算机是 Sally 的或者是 Paul 的，或者更加可能的是一台 Internet 上的未知服务器。也就是说，相对于内容来说位置无关紧要，除了它会影响性能（和合法性）。

另一个不同是一些提供内容的 Web 站点已红极一时。YouTube 是一个典型的例子。它允许用户共享他们自己创作的影片，无论任何一个可以想象的主题都能找到相应的视频。很多人都想这样做，其余的我们就是观众。所有这些都是高带宽需求的视频，据估计，直至今天的 Internet 流量中的 10% 被 YouTube 账户占用。

没有单个服务器足以强大到或者可靠到足以应付如此令人吃惊程度的需求。相反，YouTube 和其他大型内容提供商建立了它们自己的内容分发网络。这些网络使用了遍布在世界各地的数据中心，为数量极其庞大的客户群提供具有良好性能和可用性的内容。

随着时间的推移，内容分发所用技术也已经被开发了出来。早在 Web 发展初期，其流行度几乎将其毁灭。对内容的更多需求导致服务器和网络频繁地超载。很多人开始称 WWW 为全球等待（World Wide Wait）。

为了响应消费者的需求，在 Internet 核心供应了超大量的带宽，并且在网络边缘推出更快的宽带连接。带宽是提高性能的关键，但它仅仅是解决方案的一部分。为了减少无休止的延迟，研究人员还针对内容分发开发了使用带宽的不同体系结构。

一种体系结构就是内容分发网络（CDN，Content Distribution Network）。采用这种体系结构，内容提供商在 Internet 的某些位置建立一组分布式机器，然后通过它们给客户的内容。这是大牌球员的选择。另一种体系结构是对等网络（P2P，Peer-to-Peer）。采用这种体系结构，一组计算机把它们的资源集中起来，彼此互相服务提供内容，无须单独配置服务器或者任何中央控制节点。这种想法抓住了人们的想象力，因为通过共同努力一致行动，许多小球员可以团结起来形成一股巨大的冲击力。

在本节，我们将着眼于在 Internet 上分发内容面临的问题，以及实际采用的一些解决方案。在简要讨论内容的普及和 Internet 流量后，我们将介绍如何建立强大的 Web 服务器，并使用缓存来提高 Web 客户端的性能。然后我们再考查分发内容的两个主要体系结构：CDN 和 P2P 网络。正如我们将看到的那样，这两大结构在设计和性能上有很大的不同。

7.5.1 内容和 Internet 流量

为了设计和工程化工作良好的网络，我们需要理解网络运载的流量。例如，随着 Web 重心转移到了内容上，服务器已经从公司办公室迁移到了 Internet 数据中心，中心提供了大量拥有超级优秀网络连接的机器。如今要运行一个小型服务器，从 Internet 数据中心租用一台托管的虚拟服务器，甚至也比在家里或办公室运行一台与 Internet 有宽带连接真实的机器更容易和更方便。

幸运的是，关于 Internet 流量有必要了解的事实只有两个。第一个事实是，流量的变化很快，不仅表现在细节上，而且在整体上也如此。在 1994 年之前，大部分流量是传统的 FTP 文件传输（在计算机之间移动程序和数据集）和电子邮件。然后 Web 出现了，并成倍地增长。在 2000 年网络泡沫之前 Web 流量将 FTP 和电子邮件的流量远远地抛在了后面。从 2000 年左右开始，共享音乐，然后是共享电影的 P2P 文件异军突起。到 2003 年，大多数 Internet 流量变成了 P2P 流量，它把 Web 流量远远地抛在了后面。在 2000 年后期的某个时候，使用内容分发方法的网站提供视频流，比如 YouTube 等，视频流开始超过 P2P 流量。到 2014 年，思科预计所有 Internet 流量的 90% 将是这种或那种形式的视频（Cisco, 2010）。

流量并不总是那么重要。例如，虽然语音 IP 流量蓬勃发展，即使在 2003 年 Skype 开始提供服务后，但它的流量永远是图表上的一个小亮点，因为音频的带宽需求低于视频所需的两个数量级。然而，IP 语音流量强调了网络其他方面的重要性，因为它对延迟敏感。作为另一个例子，自从 2004 年 Facebook 横空出世以来，在线社交网络疯狂地生长着。2010 年，Facebook 的每天 Web 用户首次超过了谷歌用户。即使将流量放在一边不予考虑（的确有超大量的流量），在线社交网络非常重要，因为它们正在改变着人们通过 Internet 实行互动的方式。

我们必须指出的一点是 Internet 流量迅速发生着震撼般的转变，并呈现出一定的规律性。接下来会发生什么？请看本书的第 6 版，我们将告诉你答案。

有关 Internet 流量的第二个基本事实是它的高度倾斜特性。我们所熟悉的许多特性都是聚集起来取个平均值。例如，大多数成年人的平均身高接近。有一些很高的人和一些很矮的人，但很少有非常非常高或非常非常矮的人。对于这类特性，有可能设计出一个范围，它不是很大但仍然捕获了大多数人口。

Internet 流量与此不同。长期以来，人们一直认为少量 Web 站点具有大量的流量，大量的 Web 站点具有很少的流量。这个特征已经成为网络语言的一部分。早期的研究论文谈到流量时用了数据包火车（packet train）这个字眼，想法是载有大量数据包的特快列车会突然穿过链路（Jain 和 Routhier, 1986）。这种情况被正式定义为自相似（self-similarity）概念，按照我们的目的可以把它看作具有许多大大小小沟壑的网络流量，即使从不同的时间尺度来看都是不连续的一块一块（Leland 等, 1994）。后来的研究工作将大流量说成是大

象,把短流量说成是老鼠。只有少数的大象和大量的老鼠,但是大象因为庞大而举足轻重。

回到 Web 内容,相同的倾斜显而易见的。录影带出租店、公共图书馆和其他类似组织的经验表明,并非所有项目都同样受欢迎。实验中,当有 N 个电影可用,所有请求第 k 个最流行电影的比例大约为 C/K 。在这里, C 用来计算规一化的和,即

$$C = 1 / (1 + 1/2 + 1/3 + 1/4 + 1/5 + \cdots + 1/N)$$

因此,最流行电影的流行度是排名第七流行度电影的 7 倍。这个结果称为齐普夫定律 (Zipf's law) (Zipf, 1949), 它以 George Zipf 的名字命名, 这是一位哈佛大学的语言学教授, 他指出一个字在大段文字中的使用频率和它的排名成反比的。例如, 第 40 个最常见字的使用次数是第 80 个最常见字的使用次数的两倍, 是第 120 个最常见字的使用次数的 3 倍。

图 7-64(a)显示了齐普夫分布。它捕获的概念是有少数受欢迎的项目和很多不那么受欢迎的项目。要认识这种形式的分布,把数据绘制在两轴的对数尺度就容易看出,如图 7-64(b)所示。结果应该是一条直线。

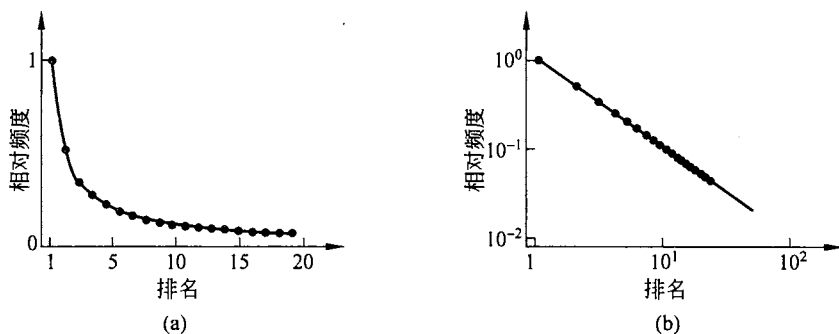


图 7-64 齐普夫分布
(a) 线性尺度; (b) 对数尺度

当人们查看流行的网页时, 结果表明它大致遵循齐普夫定律 (Breslau 等, 1999)。齐普夫分布是一个称为幂定律 (power law) 家族分布的例子。幂定律在许多人类现象上是显而易见的, 比如城市人口和财富的分布。在描述几个大牌球员和大量不知名小球员方面它们具有相同的特性, 而且它们在对数图上也表现为一条直线。人们很快发现 Internet 的拓扑结构也可大致描述为幂定律 (Faloutsos 等, 1999)。紧接着, 研究人员开始将每一个可以想象的 Internet 特性绘制成对数尺度, 在观察到一条直线后大声惊呼: “幂定律!”

然而, 比一张对数图上的直线更要紧的是这些分布对网络设计和使用的意义何在。鉴于许多形式的内容具有齐普夫或幂定律的分布特征, 似乎 Internet 上的网站普遍与齐普夫类似。反过来这意味着平均站点并不具备有用的代表性。站点最好描述成受欢迎或不受欢迎。这两种站点才是至关重要的。流行的网站明显是个问题, 因为一些热门网站可能要对 Internet 上的大多数流量负责。也许令人惊讶的是不受欢迎的网站也会是个问题。这是因为所有指向不受欢迎网站的流量总额占用了总流量的很大一部分。原因就在于有那么多不受欢迎的网站。许多不受欢迎的选择组合起来就是个问题, 这种概念已经通过书籍得到普及, 比如《长尾理论 (The Long Tail)》(Anderson, 2008a)。

类似图 7-64(a)显示衰减的曲线图很常见, 但它们各不相同。特别地, 当衰减率正比于

剩余多少材料（比如不稳定的放射性原子）时，这样的情形揭示了指数衰减（exponential decay），其下降速度远远超过齐普夫定律。在时间 t 后的项目数（比如说原子）通常表示为 $e^{-\alpha t}$ ，其中常数 α 确定了如何快速地衰变。指数衰减和齐普夫定律之间的差异在于：对于指数衰减，忽略尾部是安全的；而对于齐普夫定律，尾部的总权值很显著，不可被忽视。

为了在这个倾斜世界里有效地工作，我们必须能够构建两种类型的 Web 站。不受欢迎的网站很容易处理。使用 DNS，把许多不同的站点实际上指向 Internet 上的同一台计算机，在这台计算机上运行全部不受欢迎的网站。另一方面，流行的网站很难应付。没有单个的计算机足够强大到能运行所有流行的网站，甚至远程也没有；而且如果使用一台计算机，那么当它发生故障将使数百万用户无法访问网站。为了处理这些网站，我们必须建立内容分发系统。下面我们开始探讨这个问题。

7.5.2 服务器农场和 Web 代理

到目前为止我们看到的 Web 设计都只有一个单独的服务器，这台机器和多个客户端机器交谈。为了构建大型 Web 站点并且运行性能良好，我们可以加快服务器端的处理或者客户端的处理。在服务器端，用更强大的 Web 服务器建设成一个服务器农场，这里用一个计算机集群替代单个的服务器为客户提供服务。在客户端，可以通过更好的缓存技术来实现更好的性能。尤其是，代理缓存为一组客户提供了大容量的共享缓存。

我们将依次描述这些技术。然而，请注意，没有一种技术足以用来建立最大的 Web 站点。这些流行的站点需要用到我们在下面章节描述的内容分发，这些方法还必须与坐落在许多不同地点的计算机结合起来一起发挥作用。

服务器农场

无论一台机器的带宽多大，在超负荷之前它也只能为一定数量的 Web 请求服务。这种情况的解决方案是使用多台计算机来充当 Web 服务器。这个想法导致了如图 7-65 所示的服务器农场模型的诞生。

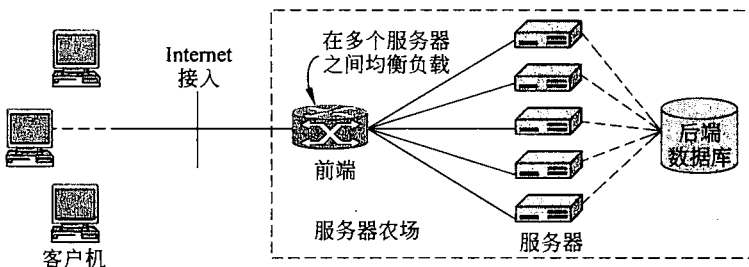


图 7-65 服务器农场

这个模型看似简单，实质上它的困难之处在于组成服务器农场的一组计算机，在客户看起来必须像单个逻辑网站。如果做不到这样，那我们只是建设了不同的网站，让这些网站并行运行而已。

如何使一组服务器以一个网站的形式出现有几种可能的解决方案。所有的解决方案都假设任何一台服务器可以处理来自任何客户端的请求。要做到这一点，每个服务器必须有

一份 Web 站点的副本。为此目的，这些服务器都连接到一个共同的后端数据库，图中用虚线表示。

一种解决方法是使用 DNS 将请求分散到服务器农场中的各个服务器。当来自一个针对 Web URL 的 DNS 请求时，DNS 服务器会返回一个服务器 IP 地址的循环列表。每个客户端尝试一个 IP 地址，通常取列表上的第一个。这个效果正如我们预期的那样，与不同客户通过不同服务器来访问相同网站的效果完全不同。DNS 方法是 CDN 的核心，我们将在本节稍后再来讨论它。

其他的解决方案是基于前端（front end）的方式，由前端把入境请求分散到服务器农场的服务器池中。当客户端使用单一目标 IP 地址联系服务器农场时，就会发生这种情况。前端通常是一个链路层交换机或 IP 路由器，也就是说，是一台处理帧或包的设备。所有的解决方案都基于该台设备（或服务器），该设备窥视网络层、传输层或应用层的头并以非标准的方式使用偷看来的信息。Web 请求和响应都通过 TCP 连接运载。为了正常工作，前端设备必须把一个请求的所有数据包分发到同一台服务器。

一个简单的设计是前端将所有的入境请求广播给所有的服务器。每个服务器按照事先的约定回答一小部分请求。例如，16 台服务器可能查看请求的源 IP 地址，只有源 IP 地址最后 4 位与它们配置的选择器相匹配时才回答该请求，其他的包都将被丢弃。虽然这样浪费了入境带宽，但往往响应包远远长于请求包，所以它并不像听起来的那样低效。

在更通用的设计中，前端可以检查数据包的 IP、TCP 和 HTTP 头，并任意地将它们映射到服务器。这种映射称为**负载均衡**（load balancing）策略，因为目标是为了均衡服务器的工作负载。这种策略可能很简单，也可能很复杂。一种简单策略是依次使用服务器，一个接着一个，或者以一种循环的形式分配服务器的使用。采用这种方法时，前端必须记住每个请求与服务器对应的映射关系，才能确保作为同一请求组成部分的随后包被发送到同一台服务器。此外，为了使网站比单个服务器更可靠，前端应注意到当某个服务器失败后，必须立即停止向它们发送请求。

与 NAT 类似，这种通用设计有点危险，或者说至少是脆弱的，因为我们刚刚创建的设备违反了分层协议的最基本原则：每一层都必须使用自己的头来实行控制目的，无论什么目的都不能检查和使用有效载荷的信息。但无论如何，人们设计了这样的系统，在未来如果因为更高层次协议的变化导致这些设备无法使用时他们或许会惊讶。在这种情况下，前端设备是一个交换机或路由器，但它根据传输层或更高层次的信息采取相应的行动。这样的盒子称为**中间件盒**（middlebox），因为它把自己插入到一条网络路径的中间，而根据协议栈的规定它是没有这项业务的。在这种情况下，前端设备最好考虑作为服务器农场内部的一部分，将所有层次都终结到应用层（因此可以使用所有这些层的头信息）。

然而，与 NAT 一样，实际上这种设计非常有用。查看 TCP 头的原因是它可以比只用 IP 信息来处理负载均衡工作得更好。例如，一个 IP 地址可能代表了整个公司，因而可能会产生许多请求。只有通过查看 TCP 或更高层次的信息，才能将这些请求映射到不同的服务器。

查看 HTTP 头的原因稍微有些不同。许多 Web 交互访问和更新数据库，比如当消费者要查看她的最近购买记录时。处理该请求的服务器将查询后台数据库。将来自同一个用户的后续请求直接指向同一台服务器是有用的，因为该服务器已经缓存了有关该用户的信息。

引起这种情况发生的最简单方法就是使用 Web Cookie（或其他区分用户的信息），而且检查 HTTP 头来找到这些 Cookie。

最后要注意的一点，尽管我们已经描述了 Web 站点的设计，同样可以用其他类型的服务器来构建服务器农场。一个例子是 UDP 之上的流媒体服务器。此时仅仅需要改变的是前端，它必须能均衡这些请求的负载（将使用与 Web 请求不同的协议头字段）。

Web 代理

使用 HTTP 发送 Web 请求和响应。在 7.3 节中，我们介绍了浏览器如何缓存响应和重用这些缓冲信息来回答未来的请求。浏览器使用各种头字段和规则来确定一个缓存的网页副本是否还很新鲜。在这里，我们不再重复这些内容。

高速缓存因为缩短了响应时间并减少了网络负载，因此能提高性能。如果浏览器可以自己判断缓存的页面是新鲜的，那么页面可以立即从缓存中提取出来，根本不会产生网络流量。然而，即使浏览器必须询问服务器来确认该页面是否依然新鲜，响应时间也缩短，网络负载也得到降低，特别对大页面尤为如此，因为只需要发送一个小的消息。

然而，浏览器可以做的最好事情是缓存用户以前访问过的所有网页。从我们关于流行网站的讨论中，你可能还记得一些热门的网页被许多人多次访问，也有很多很多不那么流行的网页。实际上，这种特性限制了浏览器缓存的有效性，因为大量的页面只被给定用户访问一次，而这些网页总是必须从服务器获取。

使缓存更有效的策略之一是在多个用户之间共享缓存。通过这种方式，为一个用户获取的一个页面可以在另一个用户发出同样请求时返回给他。没有浏览器缓存，这两个用户都需要从服务器获取该页面。当然，这种共享不适用于加密的流量和不可缓存的页面（比如，当前股票的价格），因为前者需要认证，后者是由程序返回。特别是由程序创建的动态网页，仍处于不断增长的情况，因此缓存它是无效的。不过，也有大量的网页可被许多用户访问，而且无论哪个用户发出请求看到的都是同样内容（例如，图像）。

Web 代理 (Web proxy) 是可以被多个用户共享的一个高速缓存。代理代表别人做某种行为，比如用户。有许多不同种类的代理。例如，一个 ARP 代理代表一个在别处的用户（他不能自己回答）应答 ARP 请求。Web 代理代表它的用户获取 Web 请求。它通常提供缓存的 Web 响应，而且因为它被多个用户共享，因此它有一个比所有用户浏览器都大得多的高速缓存。

当采用代理时，典型的设置是一个组织为它的所有用户运行一个 Web 代理。该组织可能是一个公司或一个 ISP。组织和个人都可以通过加快用户的 Web 请求，并减少带宽需求来获得收益。虽然独立于使用，定价对最终用户来说都是一样的，大多数公司和 ISP 根据用户所使用的带宽来收费的。

这种设置如图 7-66 所示。为了使用代理，每个浏览器必须配置成向代理而不是页面的真实服务器发出页面请求。如果代理有相应的页面，它就立刻返回页面。如果它没有，它就从服务器获取页面，并将它添加到缓存以备将来使用，同时返回给请求该页面的客户。

除了向代理而不是真正服务器发送 Web 请求外，客户还要用它的浏览器高速缓存来执行它们自己的缓存。只有在浏览器尝试以自己的高速缓存页面无法满足请求之后，才会咨询代理。也就是说，代理提供了一个第二级的缓存。

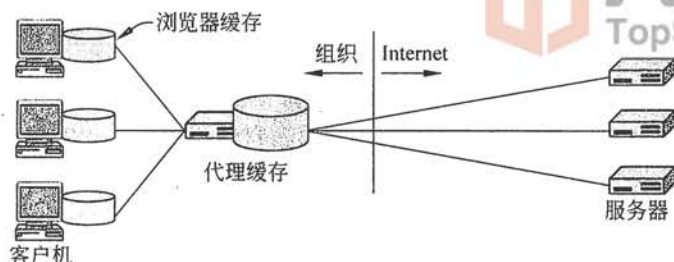


图 7-66 Web 浏览器与服务器之间的代理缓存

还可以加入进一步的代理来提供更多一级的缓存。每个代理（或浏览器）向其上行流代理（upstream proxy）发出请求。每个上行流代理为下行流代理（downstream proxy）（或浏览器）缓存页面。因此，有可能一个公司内的浏览器使用一个公司代理，而公司代理再使用一个 ISP 代理，该代理直接和 Web 服务器联系。然而，一般来说，我们给出的单级代理缓存（如图 7-66 所示）实际上也往往能获得大部分的潜在好处。相应存在的问题是流行的长尾效应。针对网站流量的研究表明，当用户数量达到一个小公司的规模（比如，100 人）时，共享缓存特别有益。随着人数增长较大，共享缓存的好处变得边缘化，由于缺乏存储空间，不流行的得不到缓存（Wolman 等，1999）。

Web 代理还提供了额外的好处，这往往是决定部署它们的一个因素。好处之一是内容过滤。管理员可以配置代理过滤掉一些被列入黑名单的网站，或者把代理配置成其他过滤器，对发出的某些请求予以过滤。例如，许多系统管理员对员工在公司上班时间观看 YouTube 视频（或更糟糕的，观看色情）会皱起眉头，因而会相应地设置它们的过滤器。用代理的另一个好处是隐私或匿名，代理作为盾牌向服务器隐匿了用户的身份。

7.5.3 内容分发网络

服务器农场和 Web 代理有助于建立大型网站，并提高 Web 性能，但对于构建一个真正流行的网站，在全球范围内提供内容服务，这些还不够。对于这些网站，需要不同的方法。

内容分发网络（CDN，Content Delivery Network）彻底扭转了传统的 Web 缓存想法。与客户在附近的缓存中寻找请求的页面副本不同，CDN 为客户提供了这样的信息：在一组分布在不同地点的节点中哪个节点拥有所请求页面的副本，并且指示客户端使用附近的节点作为服务器。

图 7-67 给出了一个 CDN 网络中数据分发所遵循的路径例子。这是一棵树。CDN 的源服务器将一份内容副本分发到 CDN 中的其他节点。在这个例子中，这些节点分别位于悉尼、波士顿和阿姆斯特丹，图中用虚线表示。然而，客户端从就近的 CDN 节点获取页面，图中用实线表示。这样，在悉尼的客户获取的都是存储在悉尼的页面副本，他们不会从源服务器处获取页面，源服务器可能在欧洲。

使用树结构有 3 个好处。首先，内容的分发具备可扩展性。随着越来越多的客户需要，可以使用更多的 CDN 节点；而且当在 CDN 节点之间分发变成瓶颈时，可以采用更多的树层次。不管有多少客户，树结构总是有效的。源服务器不会超载，因为它通过 CDN 节点

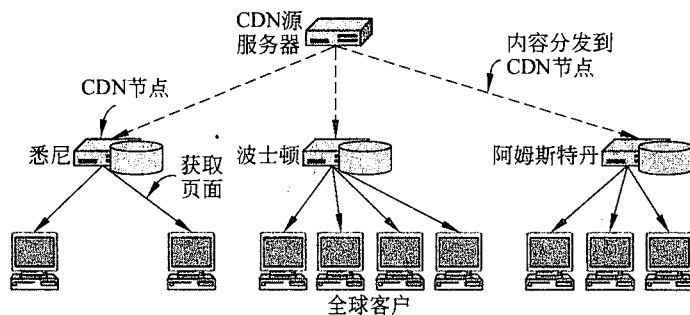


图 7-67 CDN 分发树

树和许多客户交谈；而它自己不需要应答每个页面请求。其次，每个客户都享有不错的性能，因为他是从附近的服务器而不是一个遥远的服务器那里获取页面。这是因为建立连接所需要的往返时间较短，而往返时间越短 TCP 慢启动得更快，而且较短的网络路径经过 Internet 上拥塞区域的可能性较小。最后，放置在网络上的总负荷也保持在最低水平。如果 CDN 节点都放置在恰当的位置，一个给定页面的流量应该恰好通过网络中各个部分一次。这点非常重要，最终总有人要网络带宽付费。

采用分发树的想法很直截了当。最简单的是如何组织客户使用这棵树。例如，代理服务似乎提供了一个解决方案。考查图 7-67，如果每个客户端都配置成使用悉尼、波士顿或阿姆斯特丹 CDN 节点作为缓存的 Web 代理，数据的分发将按照树的结构来进行。然而，这种策略未能在实际中奏效，原因主要有 3 个。第一个原因，网络中位于一个给定区域的客户可能属于不同的组织，所以他们可能会使用不同的 Web 代理。回想一下，大量的客户端共享缓存所获得的利益有限，而且从安全方面的考虑出发，缓存机制通常不能跨组织共享。其次，可能存在多个 CDN，但每个客户端只使用一个代理缓存。客户端应该选择哪个 CDN 作为其代理呢？最后，也许所有问题中最实际的是客户端配置了 Web 代理。他们可能会也可能不会再配置一个 CDN 来受益于内容分发，而且对此 CDN 能做的很少。

支持一棵分发树的另一种简单方法是使用镜像（mirroring）。在这种方法中，源服务器跟以前一样将内容复制给 CDN 节点。不同网络区域中的 CDN 节点称为镜像点（mirror）。源服务器上的网页包含了指向不同镜像点的显式链接，通常告诉用户这些镜像点的位置。这种设计可以让用户手动选择一个附近的镜像点来下载内容。镜像的典型应用是把一个大的软件包放在不同位置的镜像点上，例如，美国东海岸和西海岸、亚洲和欧洲。镜像站点通常是完全静态的，而且选择的站点可以保持稳定数月或数年。它们是一种经受住考验的技术。然而，它们依靠用户做内容的分发，因为镜像点实际上是不同的 Web 站点，即使它们被链接在一起。

第三种方法克服了前两种方法的困难，使用了 DNS 技术，因而称为 DNS 重定向（DNS redirection）。假设一个客户要获取 URL 为 `http://www.cdn.com/page.html` 的页面。为了提取页面，浏览器将使用 DNS 把 `www.cdn.com` 解析为一个 IP 地址。DNS 查找过程以通常的方式进行。通过使用 DNS 协议，浏览器了解到 `cdn.com` 域名服务器的 IP 地址，然后与该域名服务器联系，请求它解析 `www.cdn.com`。现在到了真正展示聪明才智的地方。该域名服务器同时作为 CDN 节点运行。它不是为每个请求返回相同的 IP 地址，而是根据发出请求的客户端 IP 地址返回不同的查询结果。答案将是最靠近客户端的 CDN 节点的 IP 地址。也

就是说，如果在悉尼的一个客户请求解析 `www.cdn.com`，域名服务器将返回悉尼 CDN 节点的 IP 地址；但如果阿姆斯特丹的一个客户端发出了同样的要求，域名服务器将返回阿姆斯特丹 CDN 节点的 IP 地址。

根据 DNS 的语义，这种策略是完全合法的。我们之前已经看到过域名服务器可能会返回变化的 IP 地址列表。在域名解析后，悉尼的客户端将直接从悉尼 CDN 节点获取页面。由于 DNS 缓存，对相同“服务器”页面的进一步请求将直接从悉尼 CDN 节点获取。整个一系列序列步骤如图 7-68 所示。

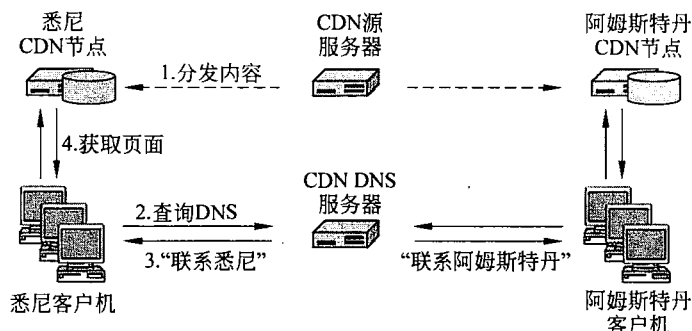


图 7-68 CDN 节点使用 DNS 将客户指向附近的 CDN 节点

上述过程中的一个复杂问题是怎样才算找到最近的 CDN 节点，以及如何到达它。要定义最近的 CDN 节点，它并不是真的和地理位置相关。在将客户端映射到某个 CDN 节点时至少有两个因素需要考虑。一个因素是网络的距离。客户端应该有一条距离短、容量高的网络路径到达 CDN 节点。这样情形下才能做到快速下载。CDN 使用一张它们以前计算的地图，把客户端的 IP 地址转换成它的网络位置。选定的 CDN 节点可能就在作为直线的最短距离上，也有可能不在最短距离上。重要的是，要综合考虑网络路径长度和途中的任何容量限制。第二个因素是 CDN 节点的当前负载。如果 CDN 节点已经超载，它们的传送反应就会缓慢，就像 Web 服务器超载一样，我们应该首先避免使用这样的服务器。因此，均衡整个 CDN 节点的负载非常有必要，应该把一些客户端映射到稍微远一点但负载轻一点的节点。

将 DNS 用作内容分发技术率先由 Akamai 在 1998 年提出的，那时 Web 在早期快速增长的负载下不堪重负。Akamai 是第一个主要的 CDN 提供商，并且成了行业的领头羊。也许比通过 DNS 将客户端连接到附近节点的想法更巧妙的是其经营的激励结构。公司给 Akamai 支付费用，以换取将它们的内容传递给客户，因此它们有客户喜欢使用的响应网站。CDN 节点必须放置在具有良好连通性的网络位置，最初这意味着在 ISP 网络内部。对于 ISP 来说，在它们的网络中放置一个 CDN 节点是有好处的，即 CDN 节点能削减它们所需要的上游网络带宽量（必须付费），就像设置代理缓存一样。此外，CDN 节点能提高 ISP 客户的响应性，使得 ISP 在客户们的眼里变得很好，从而比没有 CDN 节点的 ISP 拥有更大的竞争优势。这些好处（对 ISP 来说没有任何成本）使得安装一个 CDN 节点对 ISP 来说是简直不费吹灰之力。因此，CDN 不仅使得内容提供商、ISP 和客户都受益，而且还能赚钱。自 1998 年以来，其他公司也开始进入这个行业，所以现在 CDN 是一个有多家提供商的竞争产业。

正如这种描述所暗示的那样，多数企业没有建立它们自己的 CDN。相反，它们使用 CDN 提供商的服务来实际传送它们的内容，例如 Akamai。为了让其他公司使用 CDN 服务，我们需要添加最后一个步骤。

当 CDN 与 Web 网站所有者的代表签署内容分发的合同后，网站所有者便向 CDN 提供自己网站的内容。这个内容被推送到 CDN 节点。此外，网站所有者可以重写链接到内容的任何网页。注意，这里不是链接到其网站上的内容页面，而是通过 CDN 与内容链接的页面。为了说明这项方案是如何工作的，请考虑蓬松视频（Fluffy Video）网页的源代码，如图 7-69(a)所示。在预处理后，被转化为图 7-69(b)，并且作为 `www.fluffyvideo.com/index.html` 被放置在蓬松视频服务器。

```
<html>
<head> <title> Fluffy Video </title> </head>
<body>
<h1> Fluffy Video's Product List </h1>
<p> Click below for free samples. </p>
<a href="koalas.mpg"> Koalas Today </a> <br>
<a href="kangaroos.mpg"> Funny Kangaroos </a> <br>
<a href="wombats.mpg"> Nice Wombats </a> <br>
</body>
</html>
```

(a)

```
<html>
<head> <title> Fluffy Video </title> </head>
<body>
<h1> Fluffy Video's Product List </h1>
<p> Click below for free samples. </p>
<a href="http://www.cdn.com/fluffyvideo/koalas.mpg"> Koalas Today </a> <br>
<a href="http://www.cdn.com/fluffyvideo/kangaroos.mpg"> Funny Kangaroos </a>
<br>
<a href="http://www.cdn.com/fluffyvideo/wombats.mpg"> Nice Wombats </a> <br>
</body>
</html>
```

(b)

图 7-69

(a) 原始 Web 页面；(b) 与 CDN 链接之后的某些页面

当用户在他的浏览器上输入 URL 为 `www.fluffyvideo.com` 时，DNS 返回蓬松视频自己网站的 IP 地址，从而允许用户以正常的方式提取主页（HTML）。当用户点击该主页上的任何超链，浏览器会请求 DNS 查询 `www.cdn.com`。这次查询联系的是 CDN 的 DNS 服务器，它返回的是附近 CDN 节点的 IP 地址。然后浏览器给 CDN 节点发送一个常规 HTTP 请求，例如 `/fluffyvideo/koalas.mpg`。该 URL 标识了要返回的页面，从 `fluffyvideo` 路径开始，因此 CDN 节点可以区分它为之服务的不同公司的请求。最后，所请求的视频被返回，用户可以看到可爱的毛绒绒动物了。

把托管给 CDN 的内容和内容所有者主管的入口页面分开，隐藏在这背后的策略是让 CDN 移动大量数据的同时给内容所有者以控制权。大多数入口网页微小，只是些 HTML 文本。这些网页通常链接到大文件，比如视频和图像。正是这些大文件由 CDN 来提供服务，即使 CDN 的使用对用户完全透明。网站看起来都一样，但执行得更快。

站点们使用共享 CDN 还有另一个优势。一个 Web 站点的未来需求可能很难预测。通常情况下，潮水汹涌般的需求称为闪群（flash crowd）。当发布最新产品、有一场时装秀和发生其他事件，或者该公司卷入新闻时，这种激增的访问需求就可能发生。甚至一个以前未知的、未被访问过的落后网站也可能会突然成为 Internet 的焦点，如果它有新闻价值和被热门网站链接。由于大多数网站没有准备好处理大量激增的流量，其结果就是其中的许多网站当流量激增时立即崩溃掉。

关于这点有一个很好的案例分析。通常情况下，佛罗里达州的网站不是个繁忙的地方，虽然你可以查看有关佛罗里达州公司、公证员、文化事务，以及有关投票和选举等的信息。某种奇怪的原因，2000 年 11 月 7 日（布什与戈尔的美国总统选举日），一大堆人突然对该网站上有关选举结果的页面感兴趣。因此，突然间该网站成了世界上最繁忙的网站之一，结果自然是网站彻底崩溃。如果它早点使用 CDN，或许就会幸免于难。

通过使用 CDN，一个网站能拥有提供超大内容服务的访问能力。最大的 CDN 有数以万计的服务器部署在世界各地的不同国家。由于只有少数网站在任何一个时间（定义的）将会遇到闪群，这些网站可以使用 CDN 的能力来处理负载，直到风暴过去。也就是说，CDN 可以迅速扩大网站的服务能力。

前面的讨论只是针对 Akamai 如何工作的一个简单描述。实际上，还有更多的细节值得重视。在我们例子中描绘的 CDN 节点通常是一堆集群的机器。DNS 的重定向体现在两个层次：一个将客户映射到一个近似的网络位置，另一个将负载遍布在该位置上的节点。可靠性和性能都需要关注的两个方面。为了能够将一个客户从集群中一台机器转移到另一台机器，第二个层次上的 DNS 应答具有很短的 TTL，因此客户端很快就能重复解析。最后，虽然我们将注意力集中在分发静态对象，比如图像和视频，CDN 同样可支持动态页面创建、流媒体以及其他更多的内容。如需更多的有关 CDN 信息，请参阅（Dilley 等，2002）。

7.5.4 对等网络

不是每个人都能在世界各地设立 1000 个节点的 CDN 来分发他们的内容（实际上，租用 1000 台分布在全球各地的虚拟机并不是很难，因为有一个发展良好并且有竞争力的托管行业；但是，建立一个 CDN 仅仅是获得节点的开始）。幸运的是，对于其他人，还有一个替代品可用。它简单易用，并且可以分发大量的内容。它就是**对等网络**（P2P，Peer-to-Peer）。

P2P 网络的异军突起始于 1999 年。第一个广泛的应用是大规模的犯罪行为：50 万 Napster 用户交换没有得到版权拥有人许可的盗版歌曲，直到 Napster 在一片很大的争议声中由法院裁定被关闭。不过，对等技术是那样的有趣，并且有合法的用途。其他 P2P 系统的不断发展，吸引了用户的极大兴趣，因此 P2P 流量迅速崛起使得 Web 流量黯然失色。如今，BT（BitTorrent）是目前最流行的 P2P 协议。它的使用是如此之广泛，大家共享（授权和公共领域）视频以及其他内容，以至于它占有了所有 Internet 流量中的很大一部分。在本节我们将看到这点。

一个 P2P 文件共享网络的基本思路是将多台计算机连到一起并且把它们的资源集中起来形成一个内容分发系统。这些计算机往往是简单的家用电脑。它们并不一定是 Internet 数据中心的机器。这些计算机称为**对等节点**（peer），因为每一个都可交替充当另一个的客

户端，获取其内容；也可作为服务器，提供内容给其他对等节点。使对等系统有趣的是没有专门的基础设施，这点和 CDN 不同。每个人都参与到分发内容的任务中，而且通常还没有中央控制点。

许多人兴奋于 P2P 技术，因为它看起来就像给小家伙授予了很大的自主权。究其原因，不仅在于只有一个大型公司才能运行一个 CDN，而且任何一个人只要有一台计算机都可以加入一个 P2P 网络。正是 P2P 网络具有强大的内容分发能力，因而可以和最大的网站分庭抗礼。

考虑一个 P2P 网络，假设平均由 N 个用户组成，每个用户具有 1 Mbps 的宽带连接。P2P 网络的上载总容量，或者用户可以往 Internet 发送流量的速率是 N Mbps；而下载容量，或用户可以接收流量的速率也是 N Mbps。而且，每个用户可以同时上传和下载，因为他们每个方向上都有 1 Mbps 链路。

一个并不明显但却是真实的结论是，事实证明所有的容量都可用来高效地分发内容，甚至在所有其他用户共享一个文件副本的情况下。为了看清楚是如何做到这点的，请想象所有的用户组织成一棵二叉树，每个非叶结点的用户给其他两个用户发送。这棵树把文件的单个副本运载到所有的其他用户。为了让尽可能多的用户在任何时候（因此分发大文件的延迟较低）使用尽可能多的上传带宽，我们需要管道化用户的网络活动。试想，该文件被划分成 1000 个小片。每个用户可以从树的上方某个地方接受一个新的小片，并且在同一时间把先前收到的一小片向树的下方发送。这样，一旦管道开始后，在一部分的小片（等于树的深度）被发送出去后，所有的非叶结点用户都将忙于上传文件给其他用户。由于有大约 $N/2$ 个非叶结点的用户，该树的上传带宽是 $N/2$ Mbps。我们可以重复这一招，通过交换叶结点和非叶结点的角色，创建另一个树来使用其他 $N/2$ Mbps 的上传带宽。综合起来，这样的结构就使用了全部的容量。

这个说法意味着 P2P 网络能自我调整。它们可用的上传容量随着用户的下载需求而水涨船高。从某种意义上说，它们总是“足够大”，根本不需要任何专门的基础设施。相反，即使一个大型 Web 站点的容量也是固定的，并且要么过大，或者过小。考虑一个站点，只有 100 个集群，每个集群拥有 10 Gbps 容量。这种巨大的容量对于只有少量用户的情况并没多大帮助。该网站不能以高于 N Mbps 的速率给 N 个用户发送信息，因为速率的限制在用户这边而不在网站。而当有超过一百万个 1 Mbps 用户时，该网站又不能足够快地泵出数据让所有用户保持下载。这看起来像一个庞大的用户群，但大型 BitTorrent 网络（例如，Pirate Bay）声称有超过 10 000 000 用户。就我们的例子而言这更像 10 Tbps！

你应该对这些数字的简要分析半信半疑（或者更好，干脆不信），因为它们过于简化了情况。P2P 网络的一个重大挑战在于当面临各种各样的用户，而且用户具有不同的下载和上传容量时，如何更好地使用带宽。尽管如此，这些数字仍然表明了 P2P 的巨大潜力。

P2P 网络之所以如此重要还有另外一个原因。CDN 和其他集中运行服务的系统将提供商置于一个拥有许多用户个人信息宝库的位置，从用户的浏览喜好和网上购物，到人们的地点和电子邮件地址等悉数掌握。这些信息可以被用来提供更好以及更个性化的服务，但也可以用来侵犯人们的隐私。后者可能是故意的行为（比如作为新产品推销的手段），也可能是偶然的披露或妥协。在 P2P 系统中，不可能有单个提供商具备监测整个系统的能力。这并不意味着 P2P 系统一定会提供隐私保护，因为用户在一定程度上是相互信任的。它只

意味着 P2P 可以提供比集中式管理系统形式不同的隐私保护措施。P2P 系统现在正在探索超出文件共享的服务（例如，存储、流媒体），时间会告诉我们这样做的好处是否显著。

P2P 技术的发展遵循两条相关的路径。从更实际的角度出发，每天都有正在使用的系统。这些系统中最著名的是基于 BitTorrent 协议。在学术方面来看，一直存在着对 DHT（分布式哈希表）算法的浓厚研究兴趣，这是 P2P 系统作为整体运行性能良好的核心技术，但不需要依靠任何集中式组件。我们将分别了解这两种技术。

BitTorrent

BitTorrent 协议是 2001 年由 Brahm Cohen 开发的，目的是让一组同行们方便快捷地共享文件。有几十个免费可用的客户端运行这个协议，就好像有许多浏览器和 Web 服务器使用 HTTP 协议通信一样。BitTorrent 协议作为一个开放标准，可从 www.bittorrent.org 获得。

在一个典型的对等系统中，就像使用 BT 形成的系统，每个用户有一些可能对其他用户来说感兴趣的信息。这种信息可能是免费软件、音乐、影片、照片等。在这样的设置中共享内容必须解决 3 个问题：

- (1) 一个对等用户如何找到具有自己想下载内容的其他对等用户？
- (2) 对等用户们如何复制内容以便为大家提供高速下载？
- (3) 对等用户们如何相互鼓励上传内容给他人同时为自己下载内容？

第一个问题确实存在，因为不是所有的对等用户都拥有所有的内容，至少在最初阶段是这样的。BitTorrent 所采取的方法是为每个内容提供商创建一个内容描述，称为种子文件（torrent）。种子文件是比内容小很多的数据，被对等用户用来验证它从其他对等用户处下载的数据的完整性。想下载该内容的其他用户必须先取得种子文件，也就是说，必须找到广告内容的 Web 页面。

种子文件只是一种指定格式的文件，它包含了两类关键信息。一类信息是一个跟踪器（tracker）名字，这是一个服务器，将对等用户引导到种子文件内容。另一类信息是一个大小相等片或块（chunk）的清单，它们组成了内容。不同的种子文件可用不同大小的块，通常块的大小从 64 KB 到 512 KB 不等。种子文件包含每个块的名称，作为块的 160 位 SHA-1 哈希给出。我们将在第 8 章中介绍加密哈希，比如 SHA-1。现在，你可以把哈希想象成一个很长更安全的校验和。给定块的大小和哈希表，种子文件至少比内容小三个数量级，因此它可以被快速传送。

为了下载由种子文件所描述的内容，一个对等用户首先与该种子文件联系。跟踪器是一个服务器，它维护着一个正在主动上传和下载该内容的所有其他对等用户列表。这一组对等用户称为用户集群（swarm）。群的成员定期与跟踪器联系并向它报告自己仍然活跃，当他们离开群时也向跟踪器报告。当一个新的对等用户联系跟踪器要加入群时，跟踪器告诉它群内有那些对等用户。获取种子文件并和跟踪器联系是下载内容的前两个步骤，如图 7-70 所示。

第二个问题是如何以一种能够快速下载的方式来共享内容。当一个群首次形成时，有些对等用户必须拥有组成内容的所有块。这些对等用户就是所谓的种子（seeder）。要加入该群的其他对等用户没有块；他们是需要下载内容的对等用户。

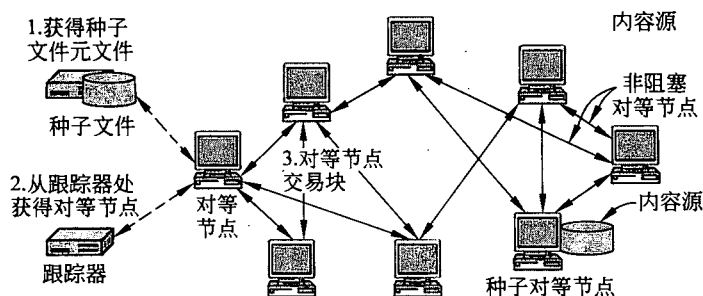


图 7-70 BitTorrent

在一个对等节点参与一个用户集群期间，它同时从其他对等节点处下载缺少的块，并且给其他对等节点上传他拥有但他们需要的块。这种交易显示在图 7-70 内容分发的最后一步。随着时间的推移，该对等节点收集到越来越多的块，直到他下载了内容的所有块。对等节点可以随时离开群（以及以后返回）。通常，一个对等节点在完成自己的下载后会停留一个短时期。随着对等节点的来来往往，用户集群的流失率相当高。

为了上面所述的方法能正常工作，每个块应该被许多对等节点可用。如果每个人都以相同顺序得到块，很可能许多对等节点都依赖于下一块的种子节点。这样容易形成瓶颈。相反，对等节点彼此之间交换各自拥有的块清单；然后，他们选择罕见的很难找到的块下载。我们的想法是下载一个难得的块将生成一个它的副本，这样使得该块更容易为其他对等节点找到并下载。如果所有的对等节点都这样做，那么经过短暂的一段时间后所有的块都将成为广泛可用。

第三个问题也许是最有趣的。CDN 节点是专门为了给用户提供服务而设置的。P2P 节点却不是。它们是用户的计算机，而且用户可能对获得一个电影比帮助其他用户下载更感兴趣。那些只想从系统中获取资源而没有实物贡献的节点称为**搭便车**（free-rider）或**吸血鬼**（leecher）。如果这样的用户太多，那么系统将无法正常工作。早期的 P2P 系统滋生了它们（Saroiu 等，2003），因此 BitTorrent 寻找一切机会来最少化它们。

BitTorrent 客户端采用的方法是奖励那些表现出良好上传行为的对等节点。每个对等节点随机采样其他对等节点，在给它们上传块的同时检索他们的块。该对等节点只与少数提供最高下载性能的对等节点们继续交易块，同时也随机尝试其他对等节点以便发现更好的合作伙伴。随机尝试对等节点们给了那些新加入的对等节点获得初始块的机会，以便他们可以与其他对等节点交易。当前正在与一个节点交换块的对等节点称为**非阻塞**（unchoked）。

随着时间的推移，这种算法趋向于匹配那些相互之间具有可比性上传和下载速率的对等节点。一个对等节点为其他对等节点作出的贡献越多，它预期的回报就越大。使用一组对等节点也有助于饱和一个对等节点的下载带宽，由此提高性能。反之，如果一个对等节点没有给其他对等节点上传块，或者做得非常缓慢，它迟早会被切断或**阻塞**（choked）。这种策略打击了反社会行为，即对等节点在群内搭便车。

抑制算法（choking algorithm）有时被描述成一种实施**针锋相对**（tit-for-tat）的策略，鼓励对等节点们反复交互合作。然而，它并不能在很大程度上防止客户游戏系统（Piatek 等，2007）。不过，注意到这个问题并且使一般用户更难以搭便车的机制有可能对 BitTorrent 的成功作出了贡献。

正如你可以从我们的讨论中看到, BitTorrent 带来了丰富的词汇。有种子文件 (torrent)、用户集群 (swarm)、吸血鬼 (leecher)、种子节点 (seeder) 以及冷落 (snubbing)、抑制 (choking)、潜伏 (lurking) 等更多其他词汇。欲了解更多信息, 请参阅有关 BitTorrent 的短文章 (Cohen, 2003) 和从 www.bittorrent.org 开始网站查询。

DHT——分布式哈希表

2000 年前后出现的 P2P 文件共享网络在研究界引发了很大的兴趣。P2P 系统的实质是避免了集中管理 CDN 结构和其他系统结构。这是一个显著的优势。集中管理的组件随着系统增长得非常大将作为瓶颈, 而且是一个单故障点。中央组件可被用来作为一个控制点 (比如关闭 P2P 网络)。但是, 早期的 P2P 系统仅仅是部分分散的, 否则如果它们完全分散就没有效率了。

我们刚才所描述的 BitTorrent 传统形式采用了对等传输, 并且每个用户集群有一个集中跟踪器 (tracker)。正是这个跟踪器成为对等系统实行分散化的最困难部分。关键问题是如何找出哪些对等节点拥有正在寻求的具体内容。例如, 每个用户可能有一个或多个数据表项, 比如歌曲、照片、程序、文件和其他用户可能想读的东西。其他用户如何找到他们想要的这些东西呢? 制作一个索引表, 即给出谁拥有什么的清单倒是简单, 但它必须是集中的。让每个对等节点保持一份它自己的索引于事无补。真正的它应该是分布式的。然而, 它需要做如此之多的工作才能保持所有对等节点的最新索引 (因为内容是在系统中移动的), 以至于这是不值得的努力。

研究界想解决的问题是在 P2P 系统中有没有可能建立一个完全分布式的但表现良好的 P2P 索引。所谓的表现良好, 我们的意思是指三件事情。首先, 每个节点只需要保留少量的有关其他节点的信息。这个特性意味着它保持最新索引的代价不会很昂贵。其次, 每个节点可以快速查看索引中的表项。否则, 它就不是个非常有用的索引。最后, 每个节点可以同时使用索引, 即使其他节点来来去去。这个属性意味着索引的性能随着节点数量的增长而越来越好。

问题的答案是: “是的”。在 2001 年发明了 4 种不同的解决方案。它们分别是 Chord (Stoica 等, 2001)、CAN (Ratnasamy 等, 2001)、Pastry (Rowstron 和 Druschel, 2001) 和 Tapestry (Zhao 等, 2004)。之后不久, 又有许多其他的解决方案被发明出来, 包括已经被实际使用的 Kademlia (Maymounkov 和 Mazieres, 2002)。该解决方案称为分布式哈希表 (DHTs, Distributed Hash Tables), 因为索引的基本功能将一个关键字映射到一个值。这简直就是一个哈希表, 当然, 解决方案是分布式的版本。

DHT 需要在节点之间的通信采用一个正规结构, 正如我们将看到的那样。这种行为有别于传统的 P2P 网络, 在传统的 P2P 系统中对等节点之间可使用任何连接。出于这个原因, DHT 称为结构化的 P2P 网络 (structured P2P networks)。传统的 P2P 协议构建的是非结构化的 P2P 网络 (unstructured P2P networks)。

我们将要描述的 DHT 解决方案是 Chord。作为一个场景, 考虑如何将 BitTorrent 中传统使用的集中式跟踪器替换成一个完全分布式的跟踪器。Chord 可以解决这个问题。在这种场景下, 总索引是所有用户集群的列表, 一台计算机可以加入到某个用户集群以便下载内容。查询索引的关键字是内容的种子文件描述, 它作为所有内容块的哈希表唯一

地标识了可以下载该内容的一个用户集群。针对每一个关键字存在哈希表中的对应值是组成该用户集群的对等节点列表。这些对等节点是要下载内容的计算机。一个想要下载内容（比如电影）的用户只有该电影的种子文件描述符。在缺乏中央数据库的情况下，DHT 必须回答的问题是如何找出一个对等节点（从 BitTorrent 数以百万计的节点中），从他那里下载电影？

Chord 的 DHT 由 n 个参与节点组成。它们在我们的场景中是运行 BitTorrent 的节点。每个节点都有一个 IP 地址，通过该地址可以联系上它。总索引扩播到这些节点。这隐含着每个节点都存储被其他节点使用的索引位和片。Chord 的关键部分是在一个虚拟空间中使用标识符来导航索引，而不是使用节点的 IP 地址或某个像电影的内容名称。从概念上讲，标识是简单的 m 位数字，它以降序的方式排列成环。

要将一个节点地址变成一个标识符，需要使用一个哈希函数 hash 将该地址映射到一个 m 位的数字。Chord 使用 SHA-1 作为哈希函数 hash。这个函数与我们在描述 BitTorrent 时提到的相同。我们将在第 8 章讨论加密时再来考查它。现在，只要知道它仅仅是一个函数，以一个可变长度的字节串作为参数，并产生一个高度随机的 160 位数字。因此，我们可以使用它把任何 IP 地址转换成一个 160 位的数字，该数字称为节点标识符（node identifier）。

在图 7-71(a)中，我们显示了 $m = 5$ 的节点标识符环（暂时忽略中间的弧线）。某些标识符对应到节点，但大多数标识符都没有节点可对应。在这个例子中，具有标识符 1、4、7、12、15、20 和 27 的节点才对应于实际节点，如图中的阴影部分所示；其余的节点都不存在实际节点。

现在，让我们把函数 $\text{successor}(k)$ 定义为环中紧跟在 k 之后的第一个实际节点的节点标识符。例如， $\text{successor}(6)=7$ 、 $\text{successor}(8)=12$ 和 $\text{successor}(22)=27$ 。

关键字（key）也可以由哈希函数产生，即用 hash（即 SHA-1）哈希一个内容名称来生成的 160 位数字。在我们的场景中，内容名称为 torrent。因此，为了将 torrent（torrent 描述文件）转换成关键字，我们计算 $\text{key}=\text{hash}(\text{torrent})$ 。这种计算只是一个调用 hash 的本地过程。

为了启动一个新的用户集群，一个节点需要将一对新的“关键字-值”（key-value）插入到索引中，该新的值对由（torrent, my-IP-address）组成。而要做到这一点，节点必须请求 $\text{successor}(\text{hash}(\text{torrent}))$ 来存储 my-IP-address。这样，该索引就被随机地分布在节点上。为了容错， p 个不同的哈希函数可以用来在 p 个节点存储该数据，但这里我们将不考虑深入的容错问题。

在 DHT 构造后的某个时间，另一个节点希望找到一个种子文件，以便它可以加入该用户集群和下载内容。节点查找种子文件的过程是这样的：首先通过哈希得到关键字 key；然后使用 $\text{successor}(\text{key})$ 找到存储了相应值的节点的 IP 地址。该值是一张该用户集群中的对等节点列表；节点可以把它自己的 IP 地址添加到列表中，并联系其他对等节点通过 BitTorrent 协议下载内容。

第一步很容易，第二步就有点困难了。为了使人们有可能找到某个特定关键字对应的节点的 IP 地址，每个节点都需要维护特定的行政管理结构数据。其中之一是沿着节点标识符组成的环，找到该节点后继节点的 IP 地址。例如，在图 7-71 中，节点 4 的后继节点是节点 7，节点 7 的后继节点是节点 12。

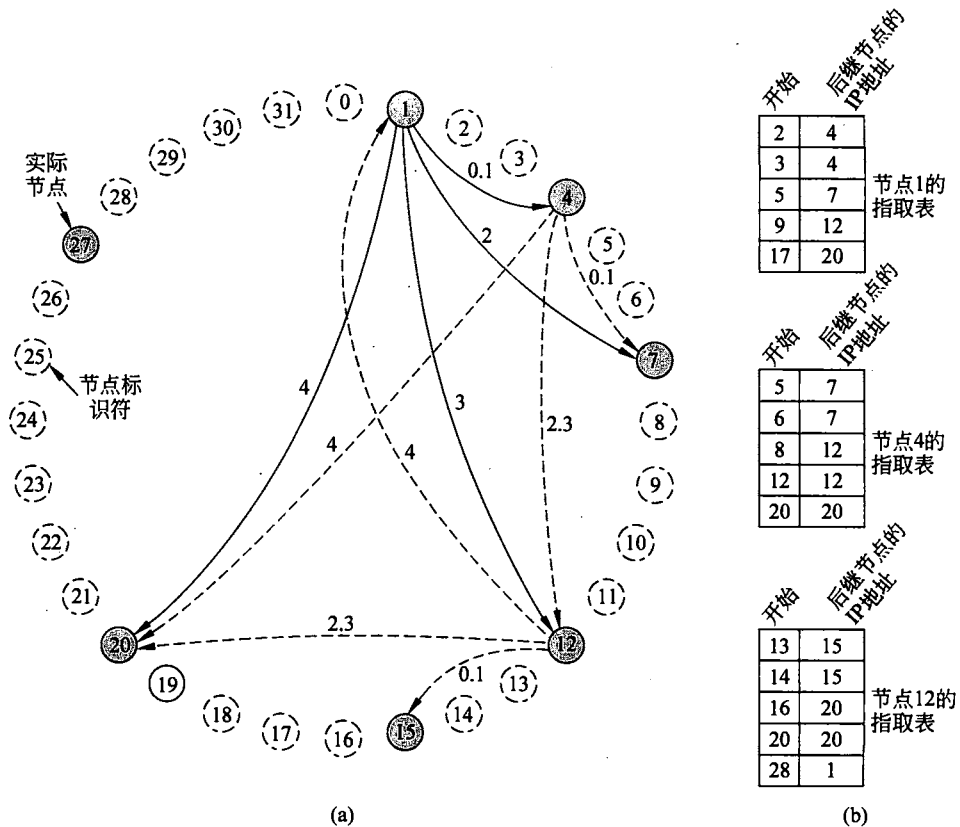


图 7-71

(a) 一组 32 个节点标识符排列成环，阴影节点对应着实际机器，弧线显示出自节点 1、节点 4 和节点 12 的指向，弧线的标记是表索引；(b) 指取表实例

现在可以如下进行查找了：请求节点向其后继节点发送一个包，该包含有其 IP 地址和它正要寻找的关键字。该包沿着环传播，直到它找到正在寻找的节点标识符的继任者。该节点检查是否有和关键字匹配的任何信息，如果有的话，它就将信息直接返回给请求节点，即它的 IP 地址。

然而，在一个大规模的对等系统中线性搜索所有节点效率非常低，因为每次搜索所需的平均节点数是 $n/2$ 。为了大大加快搜索速度，每个节点还维持一张表，该表被 Chord 称为指取表 (finger table)。指取表有 m 个表项，索引为从 0 到 $m-1$ ，每个指向一个不同的实际节点。每个表项有两个字段：start 和 successor(start) 的 IP 地址，如在图 7-71(b) 中所示的三个例子。节点标识符为 k 的节点的第 i 个表项的字段值是：

$$\text{start} = k + 2^i \quad (\text{模 } 2^m)$$

successor(start[i]) 的 IP 地址

请注意，每个节点都存储了相对数量较少的节点的 IP 地址，而且其中的大部分都相当接近节点标识符。

使用指取表，在节点 k 上查找关键字 key 的过程如下所示。如果 k 介于 k 和 $\text{successor}(k)$ 之间，那么持有 key 信息的节点就是 $\text{successor}(k)$ ，并且终止搜索；否则，搜索指取表，找到这样的表项，其 start 字段最接近 key 前任。然后，一个请求被直接发送给指取表中该表

项的 IP 地址, 请求它继续搜索。因为它接近 key, 但又低于它, 因此有很好的机会, 只需要少量的额外查询就能够返回答案。事实上, 因为每次查询都把抵达目标的剩余距离减半, 因此可以表明查找的平均次数是 $\log_2 n$ 。

作为第一个例子, 考虑在节点 1 上查询 $\text{key}=3$ 。由于节点 1 知道节点 3 位于它和它的继任者 4 之间, 因此所需的节点就是节点 4, 终止搜索, 返回节点 4 的 IP 地址。

作为第二个例子, 考虑在节点 1 上查询 $\text{key}=16$ 。因为 16 不在 1 和 4 之间, 因此查询指取表。最接近 16 的前任是 9, 因此请求被转发到节点 9 的表项的 IP 地址, 即节点 12。节点 12 自己也不知道答案, 因此它查找最接近 16 的节点, 发现是节点 14, 它产生节点 15 的 IP 地址。然后, 查询再被转发给节点 15。节点 15 观察到节点 16 位于它和它的继任者 (20) 之间, 所以它返回节点 20 的 IP 地址给调用者, 该项工作回到节点 1。

由于节点可以随时加入和离开, 因此 Chord 需要一种方法来处理这些操作。我们假设, 当系统开始运营时, 它足够小到节点之间可以直接交换信息建立第一个环和指取表。之后, 需要一个自动化过程。当一个新的节点 r 想加入时, 它必须联系一些已有的节点, 并要求它们查找自己的 $\text{successor}(r)$ 的 IP 地址。接下来, 新节点请求 $\text{successor}(r)$ 告知它的前任。然后, 新节点同时要求这两个节点在它们之间的环上插入 r 。例如, 如果在图 7-71 中节点 24 想加入, 它请求任何一个节点查询 $\text{successor}(24)$, 结果是 27。然后, 它请求节点 27, 获知自己的前任 (20)。之后它告诉这两个节点有关自己的存在, 节点 20 把节点 24 作为它的继任者, 并且节点 27 把节点 24 用作其前任。此外, 节点 27 把 21~24 范围内的关键字处理移交给节点 24, 现在属于 24 节点的管辖范围。至此, 节点 24 才算完全插入到环中。

然而, 许多指取表现在是错的。为了予以纠正, 每个节点运行一个后台进程, 该进程通过调用 successor 定期重新计算每个指取表。当这些查询结果命中一个新节点时, 相应的指取表项被更新。

当一个节点温和地离开用户集群时, 它把自己的关键字移交给其继任者, 并通知其前任自己要离开了, 以便该前任可以链接到离开节点的继任者。当一个节点出现问题崩溃时, 就会出现一个问题, 因为它的前任不再有一个有效的继任者。为了缓解这个问题, 每个节点不仅要跟踪它的直接继任者, 而且保持其 s 个直接继任者, 如果灾难发生时, 允许其跳过 $s-1$ 个连续故障节点, 并重新连接到环上。

自从 DHT 被发明出来后, 开展了大量针对它的研究。为了让你明白到底这方面的研究有多少, 让我们提出一个问题: 什么是有史以来引用最多的网络文件? 你会发现很难拿出一篇文章被引用的次数超过 Chord 文章被引用次数的一半 (Stoica 等, 2001)。尽管这方面的研究多得名副其实像座山, 但 DHT 应用才慢慢开始出现。一些 BT 客户端使用 DHT 来提供一个完全分布式跟踪器, 就是我们所描述的那种。大型商业云服务也纳入了 DHT 技术, 比如亚马逊的 Dynamo (DeCandia 等, 2007)。

7.6 本章总结

ARPANET 上的域名始于一种非常简单的方式: 一个 ASCII 文本文件列出了所有主机的名字和它们相应的 IP 地址。每天晚上, 所有机器都下载该文件。但是当 ARPANET 演变

成 Internet 并且规模发生了爆炸后，需要一个更为复杂和动态的命名方案。现在使用的是一个称为域名系统的分层计划。它将 Internet 上的机器组织成一组树。在顶层，是众所周知的通用域名，包括 com、edu 以及约 200 个国家或地区的顶级域名。DNS 作为一个分布式数据库，其服务器遍布在世界各地。通过查询 DNS 服务器，一个进程可以将一个 Internet 域名映射成 IP 地址，该地址被用来与该域的计算机通信。

电子邮件是 Internet 的原始撒手锏应用。它仍然被广泛地应用，从小孩子到老爷爷，现在每个人都在使用它。世界上绝大多数电子邮件系统都使用由 RFC 5321 和 RFC 5322 定义的协议。邮件都有简单的 ASCII 头，而且通过 MIME 可以发送许多类型的内容。邮件被提交给邮件传输代理，由它作进一步的传递；通过不同的用户代理，用户从邮件传输代理处获取并检索邮件，这些用户代理包括 Web 应用。提交的邮件使用 SMTP 发送，邮件传输的工作通过建立的一条 TCP 连接进行，即要建立一条从发送端邮件传输代理到接收端邮件传输代理的 TCP 连接。

Web 是被大多数人认为等同于 Internet 的应用程序。最初，它只是一个系统，用来无缝连接跨计算机的超文本页面（以 HTML 语言编写）。浏览器与服务器建立一个 TCP 连接并使用 HTTP 下载网页。如今，许多 Web 上的内容是动态生成的，无论是在服务器端（例如，PHP），或在浏览器中（例如，使用 JavaScript）。当与后端数据库相结合时，动态服务器页面催生出这样的 Web 应用程序，比如电子商务和搜索等。动态浏览器页面正在演变为全功能的应用，比如电子邮件，这些全功能应用运行在浏览器内，使用 Web 协议与远程服务器通信。

缓存和持续连接是被广泛用来提高 Web 性能的两种技术。尽管移动电话的带宽和处理能力都有了很大的增长，但在移动设备上使用 Web 独具挑战性。Web 站点通常给带有小屏幕的设备发送页面的裁剪版本，这样的页面具有较小图像和较少复杂的导航。

Web 协议越来越多地被用于机器对机器通信。作为对内容的描述，XML 优于 HTML，因为它更易于机器的处理。SOAP 是一个使用 HTTP 来发送 XML 消息的 RPC 机制。

自 2000 年以来，数字音频和视频已经成为 Internet 的主要驱动力。今天大多数 Internet 流量是视频，其中包含很多通过混合协议（包括 RTP/UDP 和 RTP/HTTP/TCP）来自 Web 站点的媒体流。给很多消费者现场直播媒体流，包括广播所有事件的 Internet 电台和电视台。音频和视频还可用于实时会议。许多呼叫使用了 IP 语音，而不是传统的电话网络，包括视频会议。

盛极一时的网站数量只有一小部分，而数量巨大的网站又没那么流行。为了满足流行的网站的需求，已经部署了内容分发网络。CDN 使用 DNS 把客户指向一个附近的服务器；服务器放置在世界各地的数据中心。另外，P2P 网络让一组机器彼此共享内容，比如电影。它们提供的内容分发能力可随着 P2P 网络的机器数量的不断增大而获得提升，而且可与最大的网站相媲美。